CLOUD COMPOSER SCENARIO BASED Q&A

BY - SHUBHAM WADEKAR

Copyright © Shubham Wadekar. All rights reserved.

This material is for personal use only. No part may be copied, shared, resold, or published without written permission. Unauthorized distribution is strictly prohibited and may result in action.

For permissions or licensing, contact: [shubham.p.wadekar@gmail.com]

Disclaimer: This content is for educational purposes. Accuracy is attempted but not guaranteed.

No job outcome is promised.

© Shubham Wadekar

1. A DAG has 100+ tasks, and developers struggle to maintain it. How would you redesign it?

Step 1. Identify logical groupings of tasks (by stage, data domain, or responsibility) so you can split the monolith into small, cohesive units.

Step 2. Replace a single huge DAG with multiple smaller DAGs (one per grouping) or use TaskGroup to visually and functionally group related tasks inside a DAG.

Step 3. Use dynamic DAG/task generation (factory pattern) so similar tasks are generated from configuration (YAML/JSON) instead of hand-writing each task.

Step 4. Encapsulate repeated logic into reusable operators, helper functions, or small Python callables to reduce copy-paste and make changes in one place.

Step 5. Prefer triggering or sensor-based handoffs between smaller DAGs (loose coupling) instead of creating complex intra-DAG dependencies; avoid SubDagOperator where possible.

Step 6. Add clear, consistent naming conventions, versioned DAG configs, and small readable docstrings so developers can find and understand responsibilities quickly.

Step 7. Introduce testing, linting, CI for DAGs, and observability (structured logs, metrics, alerts) so regressions are caught early and maintainability improves.

Code examples (simple patterns)

Dynamic task generation + TaskGroup example:

```
from airflow import DAG

from airflow.utils.task_group import TaskGroup

from airflow.operators.python import PythonOperator

from datetime import datetime


def do_work(item):

    print(f"processing {item}")


with DAG("example_dynamic_taskgroups", start_date=datetime(2025,1,1),
schedule_interval="@daily", catchup=False) as dag:

    stages = {

        "extract": ["src1", "src2"],

        "transform": ["tx1", "tx2", "tx3"],

        "load": ["sink1"]

    }
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
    for stage_name, items in stages.items():
        with TaskGroup(group_id=stage_name) as tg:
            for item in items:
                PythonOperator(
                    task_id=f"{stage_name}__{item}",
                    python_callable=do_work,
                    op_args=[item]
                )
```

Factory pattern (generate similar tasks from config):

```
# config.yaml -> parsed into `jobs`
# jobs = [{"id":"a","params":...}, ...]

def make_task(dag, job):
    return PythonOperator(
        task_id=f"job_{job['id']}",
        python_callable=some_callable,
        op_kwargs=job["params"],
        dag=dag
    )

with DAG("factory_dag", start_date=datetime(2025,1,1), schedule_interval="@daily") as dag:
    tasks = [make_task(dag, job) for job in jobs]
    # wire tasks programmatically as needed
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

2. A pipeline has dependencies between multiple DAGs. How would you manage cross-DAG dependencies?

Step 1. Prefer loose coupling: let one DAG signal another via an external message (Pub/Sub / Cloud Tasks / Cloud Storage marker file) or by using TriggerDagRunOperator to start the downstream DAG.

Step 2. If you need the upstream DAG to wait for downstream completion, use ExternalTaskSensor in the upstream DAG to wait for a specific downstream task or DAG run to finish — but be cautious of long-running sensors and use deferrable sensors if available.

Step 3. Use idempotent task design and unique run identifiers (execution_date or run_id passed via conf) so retries and duplicated signals don't cause inconsistent state across DAGs.

Step 4. Avoid tight circular dependencies; design a clear direction of data/control flow and, if required, use an orchestration DAG that only triggers and monitors child DAGs.

Step 5. Consider using an external orchestration/event system (Pub/Sub, Cloud Composer-triggered Cloud Functions, or Dataflow) for complex cross-DAG coordination to reduce Airflow coupling.

Step 6. Standardize the contract between DAGs (what artifacts, files, or messages indicate success) and validate this contract in tests to avoid brittle runtime failures.

Step 7. Monitor cross-DAG flows with lineage/observability (logs, metrics, and alerts) and set SLAs on critical handoffs so failures are detected and acted upon quickly.

Code examples (Trigger + ExternalTaskSensor)

Trigger downstream DAG from upstream:

```
from airflow import DAG

from airflow.operators.trigger_dagrun import TriggerDagRunOperator

from datetime import datetime


with DAG("upstream_triggerer", start_date=datetime(2025,1,1), schedule_interval="@daily",
catchup=False) as dag:

  trigger = TriggerDagRunOperator(

    task_id="trigger_downstream",

    trigger_dag_id="downstream_receiver",

    conf={"origin_run_id": "{{ run_id }}"}  # pass context if needed

  )
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Downstream DAG signals completion (example uses a final task id "downstream_done") and upstream can wait using ExternalTaskSensor:

```
from airflow import DAG
from airflow.sensors.external_task import ExternalTaskSensor
from datetime import datetime, timedelta

with DAG("upstream_waiter", start_date=datetime(2025,1,1), schedule_interval="@daily",
catchup=False) as dag:
  wait = ExternalTaskSensor(
    task_id="wait_for_downstream",
    external_dag_id="downstream_receiver",
    external_task_id="downstream_done",
    timeout=3600,        # seconds
    poke_interval=30,     # seconds
    mode="reschedule"     # use deferrable/reschedule to avoid worker slot blocking
  )
# downstream_receiver final task (example)
from airflow import DAG
from airflow.operators.empty import EmptyOperator
from datetime import datetime

with DAG("downstream_receiver", start_date=datetime(2025,1,1), schedule_interval=None,
catchup=False) as dag:
  start = EmptyOperator(task_id="start")
  # ... work ...
  done = EmptyOperator(task_id="downstream_done")
  start >> done
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Question 3: A business-critical DAG must retry failed tasks without delaying other independent tasks. How would you design it?

Step 1. Identify which tasks are business-critical and which are independent — draw a DAG diagram (or list) so dependencies are explicit.

Step 2. Decouple independent work into separate branches or separate DAGs so retries in one branch do not block unrelated branches.

Step 3. Configure retry behavior per task: set retries, retry_delay, retry_exponential_backoff, max_retry_delay on the critical tasks (not globally).

Step 4. Use sensible trigger_rule values: keep default for strict dependencies, but where you want downstream to proceed regardless of upstream retries, set trigger_rule="all_done" or use join tasks with "all_done" so independent flows continue.

Step 5. Use TaskGroup or separate smaller DAGs (triggered via TriggerDagRunOperator or an event/message) to isolate retrying logic from main flow — the main flow stays responsive while retries happen in an isolated retry DAG.

Step 6. Prefer deferrable operators or mode="reschedule" for sensors and long waits to avoid occupying worker slots during retry wait times.

Step 7. Make retrying idempotent: ensure tasks can run multiple times safely (idempotency keys, temp paths with run_id, upserts).

Step 8. Add an on_failure_callback that records failure metadata (task_id, run_id, error) to a retry queue (Pub/Sub, Cloud Tasks, DB) OR triggers a dedicated retry DAG — this lets retry attempts be managed separately and audited.

Step 9. Use pools and priority_weight to protect resources: give retry tasks a pool with limited slots so retries don't starve other tasks, or put retries in a lower-priority pool if they can wait.

Step 10. Add observability and alerts (metrics, logs, task SLAs): monitor retry rates, durations, and escalate if retries exceed thresholds so human intervention can occur before downstream impact.

Code examples (showing per-task retries, trigger_rule, and a pattern using a separate retry DAG)

Simple per-task retry and trigger_rule usage:

```
from airflow import DAG

from airflow.operators.empty import EmptyOperator

from airflow.operators.python import PythonOperator

from datetime import datetime, timedelta


def critical_work(**kwargs):

    # idempotent work; use run_id or dag_run.conf to dedupe

    print("doing critical work")

    raise Exception("force fail for demo")
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
with DAG(
    "critical_retry_isolated",
    start_date=datetime(2025,1,1),
    schedule_interval="@daily",
    catchup=False,
) as dag:

    start = EmptyOperator(task_id="start")

    critical = PythonOperator(
        task_id="critical_task",
        python_callable=critical_work,
        retries=3,                  # retry 3 times
        retry_delay=timedelta(minutes=5),
        retry_exponential_backoff=True,
        max_retry_delay=timedelta(minutes=60),
        depends_on_past=False
    )

    independent = PythonOperator(
        task_id="independent_task",
        python_callable=lambda: print("independent work runs regardless"),
        retries=0
    )

    # If you want a join that continues even if critical_task is still retrying:
    join = EmptyOperator(task_id="join_all_done", trigger_rule="all_done")

    start >> [critical, independent]
    [critical, independent] >> join
```

Pattern: trigger a separate retry-orchestration DAG on failure (conceptual)

Upstream DAG (on failure push metadata to Pub/Sub or DB, or use a TriggerDagRunOperator in an on_failure callback pattern). Practically, implement an on_failure_callback that writes to a retry queue; a small "retry-manager" DAG consumes that queue and retries tasks separately. This keeps the main DAG fast and independent.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Question 4: Orchestrate a pipeline that runs BigQuery queries, moves files in GCS, and triggers a Dataflow job which operators to use?

Step 1. Use BigQueryInsertJobOperator (or BigQueryOperator depending on provider) to run SQL jobs in BigQuery and support query jobs with job_config.

Step 2. Use GCSToGCSOperator or GCSCreateBucketOperator / GCSDeleteObjectsOperator from the Google provider to move or copy files inside GCS; GCSToGCSOperator handles rename/move between buckets/prefixes.

Step 3. Use DataflowCreatePythonJobOperator or DataflowStartFlexTemplateOperator / DataflowTemplateOperator to start Dataflow jobs (choose template vs. SDK-based operator per your job type).

Step 4. Pass outputs between steps using XComs or by writing artifacts (file paths, job IDs, manifest files) into a known GCS location and passing that path via conf to downstream tasks.

Step 5. Use BigQueryCheckOperator or BigQueryTableExistenceSensor to validate query results before moving files or triggering Dataflow.

Step 6. Make Dataflow launches idempotent by generating a unique job name with run_id and use parameters in the template conf.

Step 7. Use Sensor/operator variants with mode="reschedule" or deferrable versions to avoid worker blocking while waiting for BigQuery/Dataflow completion.

Step 8. Use retries and exponential backoff on each operator and set sensible timeouts; make sure to surface job IDs and statuses in logs for debugging.

Step 9. Secure credentials via Airflow Connections (google_cloud_default) and use service accounts with least privilege for BigQuery, GCS and Dataflow actions.

Step 10. Add monitoring: capture BigQuery job_id, Dataflow job_id, and GCS paths as XComs and send metrics/alerts on failure or long-running jobs.

Code example (BigQuery -> move GCS objects -> start Dataflow template):

```
from airflow import DAG

from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator,
BigQueryCheckOperator

from airflow.providers.google.cloud.transfers.gcs_to_gcs import GCSToGCSOperator

from airflow.providers.google.cloud.operators.dataflow import
DataflowStartFlexTemplateOperator

from airflow.operators.empty import EmptyOperator

from datetime import datetime


with DAG("bq_gcs_dataflow_pipeline", start_date=datetime(2025,1,1),
schedule_interval="@daily", catchup=False) as dag:
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
run_bq = BigQueryInsertJobOperator(
  task_id="run_bq_query",
  configuration={
    "query": {
      "query": "SELECT * FROM dataset.table WHERE dt = '{{ ds }}'",
      "useLegacySql": False,
      "destinationTable": {
        "projectId": "my-project",
        "datasetId": "staging",
        "tableId": "staged_{{ ds_nodash }}"
      },
      "writeDisposition": "WRITE_TRUNCATE"
    }
  },
)

check_bq = BigQueryCheckOperator(
  task_id="check_bq_results",
  sql="SELECT COUNT(1) FROM `my-project.staging.staged_{{ ds_nodash }}` WHERE TRUE",
  use_legacy_sql=False
)

move_to_processed = GCSToGCSOperator(
  task_id="move_files_in_gcs",
  source_bucket="my-bucket-temp",
  source_object="outputs/{{ ds }}/**",
  destination_bucket="my-bucket-processed",
  destination_object="outputs/{{ ds }}/",
  move_object=True
)
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
start_dataflow = DataflowStartFlexTemplateOperator(
  task_id="start_dataflow",
  project_id="my-project",
  region="us-central1",
  body={
    "launchParameter": {
      "jobName": "dataflow-job-{{ ds_nodash }}",
      "containerSpecGcsPath": "gs://my-templates/flex-template.json",
      "parameters": {
        "input": "gs://my-bucket-processed/outputs/{{ ds }}/",
        "output": "gs://my-bucket-processed/results/{{ ds }}/"
      }
    }
  }
)

done = EmptyOperator(task_id="done")

run_bq >> check_bq >> move_to_processed >> start_dataflow >> done
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Question 5: An external API must be called inside your workflow. How to integrate it with Cloud Composer?

Step 1. Add the API credentials to Airflow Connections (Connection type HTTP or use a Secrets Manager) rather than hardcoding keys; use Variable or Secret backend for non-connection secrets.

Step 2. Use the SimpleHttpOperator or HttpHook for straightforward REST calls (supports method, endpoint, headers, payload). For complex logic use PythonOperator with requests and HttpHook.

Step 3. Implement retries and exponential backoff at the operator level (retries, retry_delay) and optionally use tenacity inside a Python callable for fine-grained control.

Step 4. Respect API rate limits: implement rate limiting, a throttling mechanism, or use a queue (Pub/Sub/Cloud Tasks) to smooth spikes.

Step 5. Make calls idempotent and pass an idempotency key (or use upsert semantics) so retries won't create duplicates.

Step 6. Use sensors (HttpSensor) for polling endpoints when you need to wait for an external condition; use deferrable sensors if available.

Step 7. Handle auth cleanly: OAuth flows can be implemented in a one-time task that stores tokens in Connections/Secrets; use refresh token logic in hook code.

Step 8. Capture responses and errors in structured logs and optionally write success markers (to GCS/DB) so downstream tasks consume well-known artifacts.

Step 9. If latency or reliability is a concern, decouple the call by pushing a message to a queue (Pub/Sub/Cloud Tasks) and have a separate service or DAG worker process the queue and call the API asynchronously.

Step 10. Add tests (mock the API in unit tests), and run integration tests in staging to validate retries, backoff, auth, and idempotency before production.

Code examples (SimpleHttpOperator and a PythonOperator with HttpHook + backoff)

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

SimpleHttpOperator usage:

```
from airflow import DAG
from airflow.providers.http.operators.http import SimpleHttpOperator
from datetime import datetime, timedelta

with DAG("call_external_api_simple", start_date=datetime(2025,1,1),
schedule_interval="@daily", catchup=False) as dag:
  call_api = SimpleHttpOperator(
    task_id="call_api",
    method="POST",
    http_conn_id="my_api_conn",   # configured in Airflow Connections
    endpoint="/v1/process",
    data='{"key":"value"}',
    headers={"Content-Type": "application/json"},
    response_check=lambda response: response.status_code == 200,
    retries=3,
    retry_delay=timedelta(seconds=30)
  )
```

PythonOperator with HttpHook and tenacity-style backoff (example):

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.http.hooks.http import HttpHook
from datetime import datetime, timedelta
import time

def call_api_with_backoff(**kwargs):
  hook = HttpHook(method="POST", http_conn_id="my_api_conn")
  payload = {"key":"value"}
  max_attempts = 5
  backoff = 5
  for attempt in range(1, max_attempts + 1):
    response = hook.run(endpoint="/v1/process", json=payload)
    if 200 <= response.status_code < 300:
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
        return response.json()
    if attempt == max_attempts:
        response.raise_for_status()
    time.sleep(backoff * attempt)  # simple linear backoff


with DAG("call_external_api_python", start_date=datetime(2025,1,1),
schedule_interval="@daily", catchup=False) as dag:
  call_api = PythonOperator(
    task_id="call_api_python",
    python_callable=call_api_with_backoff,
    retries=0  # handled inside callable
  )
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

6. A business requirement is to connect to an on-prem SQL Server database. How would you implement this with hooks?

Step 1. Set up network connectivity between Cloud Composer and the on-prem SQL Server (VPN, Interconnect, or a secure proxy).

Step 2. Configure firewall rules to allow Composer workers to reach the SQL Server host/port securely.

Step 3. Create a new Airflow Connection in the UI of type "MSSQL" with host, port, schema, user, password.

Step 4. Use the MsSqlHook from airflow.providers.microsoft.mssql.hooks.mssql for database interactions.

Step 5. Use operators like MsSqlOperator for running queries or PythonOperator with MsSqlHook for custom logic.

Step 6. Ensure credentials are stored securely in Airflow Connections or Secret Manager instead of hardcoding.

Step 7. Add retries and backoff for transient connectivity issues.

Step 8. Test connectivity with a small "SELECT 1" before production usage.

Step 9. Monitor query duration and errors, and set timeouts for long queries.

Step 10. If large data transfers are required, stage data into GCS/BigQuery instead of moving heavy loads directly.

Code using MsSqlHook:

```
from airflow import DAG

from airflow.providers.microsoft.mssql.hooks.mssql import MsSqlHook

from airflow.operators.python import PythonOperator

from datetime import datetime


def query_mssql(**kwargs):

  hook = MsSqlHook(mssql_conn_id="my_mssql_conn")

  records = hook.get_records("SELECT TOP 10 * FROM my_table")

  print(records)


with DAG("mssql_example", start_date=datetime(2025,1,1), schedule_interval="@daily",
catchup=False) as dag:

  run_query = PythonOperator(

    task_id="run_query",

    python_callable=query_mssql

  )
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

7. You must load daily CSV files from GCS into a partitioned BigQuery table. How would you orchestrate this pipeline?

Step 1. Schedule the DAG daily, aligned with when CSV files are available in GCS.

Step 2. Use a GCS sensor (GCSObjectsWithPrefixExistenceSensor) to wait for the file to land.

Step 3. Use the GCSToBigQueryOperator to load CSV files directly into BigQuery.

Step 4. Configure the operator with autodetect=True or provide schema, set write_disposition to "WRITE_APPEND".

Step 5. Use the parameter time_partitioning in GCSToBigQueryOperator to load into the correct partition (for example by ingestion date or file date).

Step 6. Use Jinja templates to dynamically point to daily file paths like gs://bucket/data/{{ ds }}/file.csv.

Step 7. Validate load with a BigQueryCheckOperator to ensure rows are inserted.

Step 8. Add retries and backoff for transient GCS or BigQuery issues.

Step 9. Add monitoring on load size and partition usage.

Step 10. Document partitioning logic clearly so downstream queries know how data is organized.

Code with GCSToBigQueryOperator:

```
from airflow import DAG

from airflow.providers.google.cloud.transfers.gcs_to_bigquery import GCSToBigQueryOperator

from airflow.providers.google.cloud.sensors.gcs import
GCSObjectsWithPrefixExistenceSensor

from datetime import datetime


with DAG("gcs_to_bq_partitioned", start_date=datetime(2025,1,1), schedule_interval="@daily",
catchup=False) as dag:


  wait_for_file = GCSObjectsWithPrefixExistenceSensor(

    task_id="wait_for_file",

    bucket="my-data-bucket",

    prefix="daily/{{ ds }}/"

  )
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
load_csv = GCSToBigQueryOperator(
    task_id="load_to_bq",
    bucket="my-data-bucket",
    source_objects=["daily/{{ ds }}/file.csv"],
    destination_project_dataset_table="myproject.mydataset.mytable${{ ds_nodash }}",
    source_format="CSV",
    write_disposition="WRITE_APPEND",
    autodetect=True,
    time_partitioning={"type": "DAY"}
)

wait_for_file >> load_csv
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

8. A query needs to rebuild a summary table every night. How would you automate this?

Step 1. Create a DAG scheduled daily at night (for example midnight).

Step 2. Use BigQueryInsertJobOperator to run the SQL query that rebuilds the summary table.

Step 3. Configure the query with destinationTable pointing to the summary table.

Step 4. Set writeDisposition="WRITE_TRUNCATE" so the table is replaced with fresh data.

Step 5. Keep useLegacySql=False to run standard SQL.

Step 6. Make sure the query is optimized and tested separately before automation.

Step 7. Add retries and sensible timeouts in case of BigQuery service issues.

Step 8. Add BigQueryCheckOperator to verify the summary table has expected row count.

Step 9. Monitor run duration and cost, set alerts if query exceeds threshold.

Step 10. Document schedule and logic for maintainability.

Code with BigQueryInsertJobOperator:

```
from airflow import DAG

from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator,
BigQueryCheckOperator

from datetime import datetime


with DAG("rebuild_summary_table", start_date=datetime(2025,1,1), schedule_interval="0 0 * *
*", catchup=False) as dag:


  rebuild = BigQueryInsertJobOperator(

    task_id="rebuild_summary",

    configuration={

      "query": {

        "query": """

          SELECT region, SUM(sales) as total_sales

          FROM `myproject.mydataset.transactions`

          WHERE DATE(order_date) = '{{ ds }}'

          GROUP BY region

        """,

        "destinationTable": {

          "projectId": "myproject",

          "datasetId": "analytics",

          "tableId": "daily_summary"
```

© Shubham Wadekar

```
            },
            "writeDisposition": "WRITE_TRUNCATE",
            "useLegacySql": False
          }
        }
    )

    check = BigQueryCheckOperator(
        task_id="check_summary",
        sql="SELECT COUNT(1) FROM `myproject.analytics.daily_summary` WHERE
DATE(_PARTITIONTIME) = '{{ ds }}'",
        use_legacy_sql=False
    )

    rebuild >> check
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

9. A schema change in source data is causing BigQuery load failures. How would you modify your Composer DAG to handle this?

Step 1. Detect schema changes early — add a lightweight pre-load validation task that samples incoming files (peek header/JSON keys) and compares to the expected schema or a stored schema registry.

Step 2. Route raw incoming files into a staging/raw table (or a separate raw GCS prefix) instead of loading directly into the production table so schema drift doesn't break consumers.

Step 3. Use BigQuery load options that allow schema evolution where appropriate (schemaUpdateOptions like ALLOW_FIELD_ADDITION / ALLOW_FIELD_RELAXATION) when loading into a staging table.

Step 4. If automatic schema update is not acceptable, create a transformation step that normalizes incoming schema to the canonical schema (fill missing columns, cast types, drop unexpected fields) before final load.

Step 5. Implement a merge step (MERGE or INSERT/UPDATE) from staging into the production partitioned table so you can control how new fields are handled and keep final table stable.

Step 6. Add alerts and an automated approval path: when a schema change is detected, notify owners (Slack/email) and optionally pause automatic promotion until a human approves schema changes.

Step 7. Keep the load idempotent: use run_id / file fingerprinting to avoid duplicate ingestion if you retry with changed schema handling.

Step 8. Add robust logging and persist the problematic file and schema diff to GCS/BigQuery so engineers can replay and debug.

Step 9. Add unit/integration tests for schema evolution (sample files with new fields) and run them in CI before enabling auto-promotion to prod.

Step 10. Document the schema evolution policy and update the schema registry or contract so downstream consumers know how changes are handled.

Code: BigQuery load with schema update into staging, then MERGE to prod

```
from airflow import DAG

from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator

from airflow.operators.python import PythonOperator

from datetime import datetime


def generate_merge_query(staging_table, prod_table):

    return f"""

    MERGE `{prod_table}` T

    USING `{staging_table}` S

    ON T.id = S.id

    WHEN MATCHED THEN
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
  UPDATE SET T.col1 = S.col1, T.col2 = S.col2
WHEN NOT MATCHED THEN
  INSERT ROW
"""


with DAG("bq_schema_evolution", start_date=datetime(2025,1,1), schedule_interval="@daily",
catchup=False) as dag:


  # Load into staging allowing new fields to be added
  load_staging = BigQueryInsertJobOperator(
    task_id="load_to_staging",
    configuration={
      "load": {
        "sourceUris": ["gs://my-bucket/incoming/{{ ds }}/file-*.json"],
        "destinationTable": {
          "projectId": "myproject",
          "datasetId": "staging",
          "tableId": "incoming_raw_{{ ds_nodash }}"
        },
        "sourceFormat": "NEWLINE_DELIMITED_JSON",
        "writeDisposition": "WRITE_TRUNCATE",
        # allow new fields to be added in staging
        "schemaUpdateOptions": ["ALLOW_FIELD_ADDITION", "ALLOW_FIELD_RELAXATION"],
        "autodetect": True
      }
    }
  )


  # Merge from staging into production canonical table
  merge = BigQueryInsertJobOperator(
    task_id="merge_to_prod",
    configuration={
      "query": {
        "query": generate_merge_query(
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
            staging_table="myproject.staging.incoming_raw_{{ ds_nodash }}",
            prod_table="myproject.analytics.prod_table"
          ),
          "useLegacySql": False,
          "writeDisposition": "WRITE_APPEND"
        }
      }
  )

  load_staging >> merge
```

10. You need to trigger a parameterized Dataflow job daily with different input files. How would you design the DAG?

Step 1. Use Dataflow templates (Flex or Classic) so you can start jobs by passing parameters (input path, output path, jobName) rather than building and launching pipeline code each run.

Step 2. Make a discovery step that lists or computes the input files for today's run (list GCS prefix or read manifest). This step returns a list of file paths.

Step 3. For each input file (or logical group), dynamically create a Dataflow start task that passes the file path as a parameter to the template. Use task mapping or loop-based dynamic task generation.

Step 4. Ensure each Dataflow job has a unique jobName that includes ds_nodash or run_id to avoid collisions and make tracking easier.

Step 5. Use DataflowStartFlexTemplateOperator or DataflowStartTemplateOperator (depending on template type) to launch the jobs and capture job_id in XCom for monitoring.

Step 6. Add a monitoring/sensor step per job (or a consolidated watcher) that polls Dataflow job status and fails the DAG if a critical job fails.

Step 7. Make launches idempotent by checking whether a job for the same input/run already exists and skip if found (use Dataflow API via hook if needed).

Step 8. Add retries/backoff for transient Dataflow submission failures and limit concurrent Dataflow runs if quotas are a concern.

Step 9. Secure template and parameter values through Airflow connections/variables and validate input paths before triggering.

Step 10. Collect job metadata (job_id, start_time, input_path) into a log table or GCS manifest for auditing and easier restarts.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Code: Launch multiple parameterized Dataflow flex-template jobs dynamically

```
from airflow import DAG

from airflow.operators.python import PythonOperator

from airflow.providers.google.cloud.operators.dataflow import
DataflowStartFlexTemplateOperator

from datetime import datetime


def discover_inputs(**kwargs):

  # return list of GCS URIs for today's inputs

  return [

    "gs://my-bucket/inputs/{{ ds }}/fileA.csv",

    "gs://my-bucket/inputs/{{ ds }}/fileB.csv",

  ]


with DAG("dataflow_param_jobs", start_date=datetime(2025,1,1), schedule_interval="@daily",
catchup=False) as dag:


  discover = PythonOperator(

    task_id="discover_inputs",

    python_callable=discover_inputs,

    provide_context=True

  )


  # dynamically create tasks

  inputs = ["fileA", "fileB"]  # in practice you would pull from XCom of discover task

  for idx, input_name in enumerate(inputs):

    start_job = DataflowStartFlexTemplateOperator(

      task_id=f"start_dataflow_{input_name}",

      project_id="myproject",

      region="us-central1",

      body={

        "launchParameter": {

          "jobName": f"my-dataflow-job-{input_name}-{{{{ ds_nodash }}}}",

          "containerSpecGcsPath": "gs://my-templates/flex-template.json",

          "parameters": {
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
            "input": f"gs://my-bucket/inputs/{{{{ ds }}}}/{input_name}.csv",
            "output": f"gs://my-bucket/outputs/{{{{ ds }}}}/{input_name}/"
        }
    }
  }
)
discover >> start_job
```

11. A Dataflow job triggered from Composer is failing halfway. How would you capture its status and retry?

Step 1. Capture the Dataflow job_id at launch using the Dataflow operator's return value or XCom so you can reference that job for status checks and retries.

Step 2. Use a Dataflow job status sensor (or poll the Dataflow API via DataflowHook) to monitor the job until it reaches a terminal state (JOB_STATE_DONE, JOB_STATE_FAILED, JOB_STATE_CANCELLED).

Step 3. On failure, gather diagnostic info: job messages, worker logs, and stack traces (use Dataflow REST API or Stackdriver/Cloud Logging) and persist the details to GCS/BigQuery for post-mortem.

Step 4. Implement an automated retry strategy: small number of restarts with exponential backoff; if transient error, resubmit the pipeline with same parameters and unique jobName.

Step 5. For streaming jobs, consider snapshot/restore or use checkpointing via Pub/Sub offsets or Bigtable/State for exactly-once semantics — use these to avoid reprocessing duplication when retrying.

Step 6. If the job repeatedly fails, escalate and stop automatic retries; surface a ticket or pager alert to engineers with job metadata.

Step 7. Make the resubmit logic idempotent: check if partial output exists and either resume or clean up before re-run, depending on pipeline semantics.

Step 8. Optionally, create a retry-manager DAG that receives failed-job metadata and orchestrates restarts so the main DAG remains simple.

Step 9. Include configurable failure thresholds and human-in-the-loop approval for restarts when data or cost impact is high.

Step 10. Instrument metrics (failure reason histogram, restart counts) and dashboards so you can spot systemic issues early.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Code: Start Dataflow, monitor via sensor, and resubmit on failure

```
from airflow import DAG

from airflow.providers.google.cloud.operators.dataflow import
DataflowStartFlexTemplateOperator

from airflow.providers.google.cloud.sensors.dataflow import DataflowJobStatusSensor

from airflow.operators.python import PythonOperator

from datetime import datetime, timedelta


def handle_failure(context):

  job_id = context['ti'].xcom_pull(task_ids='start_dataflow', key='job_id')

  # persist diagnostics or send alert

  print(f"Dataflow job failed: {job_id}")


with DAG("dataflow_monitor_and_retry", start_date=datetime(2025,1,1),
schedule_interval="@daily", catchup=False) as dag:


  start = DataflowStartFlexTemplateOperator(

    task_id="start_dataflow",

    project_id="myproject",

    region="us-central1",

    body={

      "launchParameter": {

        "jobName": "dataflow-job-{{ ds_nodash }}",

        "containerSpecGcsPath": "gs://my-templates/flex-template.json",

        "parameters": {"input": "gs://my-bucket/inputs/{{ ds }}/"}

      }

    }

  )


  watch = DataflowJobStatusSensor(

    task_id="watch_dataflow",

    project_id="myproject",

    region="us-central1",

    job_id="{{ task_instance.xcom_pull(task_ids='start_dataflow', key='job_id') }}",

    poke_interval=60,

    timeout=60 * 60 * 6  # 6 hours
```

© Shubham Wadekar

```
)

# handle failure via sensor/operator failure callbacks; example shows on_failure set in default args or task

start >> watch
```

12. Your pipeline must choose between running a batch Dataflow job or skipping it based on data availability. How would you implement this logic?

Step 1. Add a lightweight data-availability check as the first task (e.g., GCSObjectsWithPrefixExistenceSensor or a PythonOperator that lists GCS prefix or checks a manifest).

Step 2. Use a BranchPythonOperator (or ShortCircuitOperator) to choose the path: run Dataflow or skip/mark as no-op.

Step 3. If branching, return the task_id of the Dataflow-start task when data exists, otherwise return the task_id of a "skip" downstream task.

Step 4. Ensure downstream tasks handle the skipped branch correctly (use Dummy/EmptyOperator as the "no-op" and then join with a trigger_rule like "one_success" or "none_failed_min_one_success" when combining branches).

Step 5. Keep the Dataflow launching logic idempotent and parameterized so it only runs when necessary.

Step 6. Add observability: log which branch ran, number of files found, and whether Dataflow executed or skipped.

Step 7. If the decision is time-sensitive, use deferrable sensors to avoid worker slot consumption while waiting for data.

Step 8. Record metrics for skipped runs vs runs executed so you can tune upstream producers and set SLAs.

Step 9. If skipping should trigger downstream notifications or alternative flows, implement them in the "skip" branch (e.g., notify stakeholders).

Step 10. Test the branching logic with unit tests and in staging using mock/no-data scenarios.

Code: Branching to run Dataflow or skip

```
from airflow import DAG

from airflow.operators.python import BranchPythonOperator, PythonOperator

from airflow.providers.google.cloud.operators.dataflow import
DataflowStartFlexTemplateOperator

from airflow.operators.empty import EmptyOperator

from airflow.providers.google.cloud.sensors.gcs import
GCSObjectsWithPrefixExistenceSensor

from datetime import datetime
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
def choose_branch(**kwargs):
    files = kwargs['ti'].xcom_pull(task_ids='check_data')
    if files:
        return 'start_dataflow'
    else:
        return 'skip_dataflow'

def check_files():
    # return list-like truthy if files exist (replace with real GCS listing)
    return []

with DAG("conditional_dataflow", start_date=datetime(2025,1,1), schedule_interval="@daily",
catchup=False) as dag:

    check = PythonOperator(
        task_id="check_data",
        python_callable=check_files
    )

    branch = BranchPythonOperator(
        task_id="decide",
        python_callable=choose_branch,
        provide_context=True
    )

    start_dataflow = DataflowStartFlexTemplateOperator(
        task_id="start_dataflow",
        project_id="myproject",
        region="us-central1",
        body={
            "launchParameter": {
                "jobName": "conditional-dataflow-{{ ds_nodash }}",
                "containerSpecGcsPath": "gs://my-templates/flex-template.json",
                "parameters": {"input": "gs://my-bucket/inputs/{{ ds }}/"}
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
        }
    }
)

skip = EmptyOperator(task_id="skip_dataflow")
join = EmptyOperator(task_id="join", trigger_rule="one_success")

check >> branch
branch >> [start_dataflow, skip]
[start_dataflow, skip] >> join
```

13. A PySpark job must run daily on Dataproc, and the cluster should be deleted after completion. How would you orchestrate it?

Step 1. Schedule a daily DAG that clearly scopes the job run (use schedule aligned to business needs and set catchup=False unless backfills required).

Step 2. Create an ephemeral Dataproc cluster at the start of the DAG (use DataprocCreateClusterOperator or a workflow/template that creates the cluster).

Step 3. Submit the PySpark job to the newly-created cluster using the Dataproc submit operator (capture the returned job_id/XCom for monitoring).

Step 4. Monitor the Dataproc job until it reaches a terminal state (use the operator's built-in wait or a Dataflow/Dataproc sensor if needed).

Step 5. Ensure the cluster-delete task runs regardless of job success or failure by setting the delete operator's trigger_rule to "all_done" (so cluster teardown always executes).

Step 6. Use preemptible worker nodes and appropriate machine types for cost savings, and set autoscaling or worker count tuned for the job to minimize runtime cost.

Step 7. Make the job idempotent and use run identifiers (run_id or ds) in temporary paths/output so retries or partial runs do not corrupt results.

Step 8. Persist job metadata (job_id, cluster_name, start/end times, exit code) to GCS/BigQuery for auditing and troubleshooting.

Step 9. Add retries & backoff only for submission/creation steps where safe; avoid automatic re-creation loops that could cost money—fail fast and notify on systemic issues.

Step 10. Optionally use Dataproc Workflow Templates (instantiate per-run) to simplify creation -> job -> delete patterns and to centralize cluster/job configuration.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Dataproc ephemeral cluster DAG example using DataprocSubmitPySparkJobOperator and DataprocDeleteClusterOperator:

```
from airflow import DAG
from airflow.providers.google.cloud.operators.dataproc import (
    DataprocCreateClusterOperator,
    DataprocSubmitPySparkJobOperator,
    DataprocDeleteClusterOperator
)
from airflow.operators.empty import EmptyOperator
from datetime import datetime, timedelta

CLUSTER_NAME = "ephemeral-cluster-{{ ds_nodash }}"
PROJECT_ID = "my-gcp-project"
REGION = "us-central1"
ZONE = "us-central1-a"
PYSPARK_URI = "gs://my-bucket/jobs/my_pyspark_job.py"

default_args = {
    "owner": "data-eng",
    "depends_on_past": False,
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    "dataproc_pyspark_ephemeral",
    start_date=datetime(2025,1,1),
    schedule_interval="@daily",
    catchup=False,
    default_args=default_args,
) as dag:

    start = EmptyOperator(task_id="start")
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
create_cluster = DataprocCreateClusterOperator(
    task_id="create_cluster",
    project_id=PROJECT_ID,
    cluster_name=CLUSTER_NAME,
    region=REGION,
    cluster_config={
        "gce_cluster_config": {"zone_uri": ZONE},
        "master_config": {"num_instances": 1, "machine_type_uri": "n1-standard-4"},
        "worker_config": {
            "num_instances": 2,
            "machine_type_uri": "n1-standard-4",
            "num_preemptible_instances": 2
        },
        # optional autoscaling / image version etc.
    },
)

submit_pyspark = DataprocSubmitPySparkJobOperator(
    task_id="submit_pyspark",
    project_id=PROJECT_ID,
    region=REGION,
    main=PYSPARK_URI,
    cluster_name=CLUSTER_NAME,
    # optional: dataproc_pyspark_properties, args, jar_uris, pyfiles, etc.
)

# ensure delete runs even if submit_pyspark fails
delete_cluster = DataprocDeleteClusterOperator(
    task_id="delete_cluster",
    project_id=PROJECT_ID,
    cluster_name=CLUSTER_NAME,
    region=REGION,
    trigger_rule="all_done",  # crucial: always tear down cluster
)
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
end = EmptyOperator(task_id="end")


start >> create_cluster >> submit_pyspark >> delete_cluster >> end
```

14. You need to run a Hive query after a Spark job finishes on Dataproc. How would you chain these in a Composer DAG?

Step 1. Run the Spark job on the same (ephemeral or persistent) Dataproc cluster and capture its completion (use Dataproc submit operator which waits until the job finishes).

Step 2. After the Spark job task, submit the Hive query via DataprocSubmitJobOperator configured with a hiveJob payload that points to the Hive script or the query text.

Step 3. If using an ephemeral cluster, ensure the cluster is created before the Spark step and deleted only after the Hive step completes (use trigger_rule="all_done" on delete task).

Step 4. Use task dependencies to enforce order: create_cluster >> submit_spark >> submit_hive >> delete_cluster.

Step 5. Pass artifacts/paths between steps via XCom or by writing to known GCS locations (Spark writes results to a Hive-accessible location or to HDFS/GCS path that Hive can read).

Step 6. Make Hive steps idempotent (use CREATE OR REPLACE TABLE or write to partitioned location by ds) to handle retries safely.

Step 7. For heavy chaining or many jobs, consider using a Dataproc Workflow Template that includes both Spark and Hive jobs to let Dataproc manage ordering and cluster lifecycle.

Step 8. Monitor both Spark and Hive job exit statuses and collect logs (gcloud/dataproc logs or Cloud Logging) for debugging.

Step 9. If the Hive job depends on the Spark output consistency, add a lightweight verification step (e.g., BigQuery/metadata check or file existence check) before starting Hive.

Step 10. Document where intermediate data is stored and the schema expected by Hive to make troubleshooting and future changes easier.

Chained Spark then Hive example using DataprocSubmitPySparkJobOperator and a generic DataprocSubmitJobOperator (hiveJob):

```
from airflow import DAG

from airflow.providers.google.cloud.operators.dataproc import (

  DataprocCreateClusterOperator,

  DataprocSubmitPySparkJobOperator,

  DataprocSubmitJobOperator,

  DataprocDeleteClusterOperator

)

from airflow.operators.empty import EmptyOperator

from datetime import datetime, timedelta
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
CLUSTER_NAME = "chain-cluster-{{ ds_nodash }}"
PROJECT_ID = "my-gcp-project"
REGION = "us-central1"
PYSPARK_URI = "gs://my-bucket/jobs/my_pyspark_job.py"
HIVE_SCRIPT_GCS = "gs://my-bucket/jobs/hive_transform.sql"

with DAG(
    "dataproc_spark_then_hive",
    start_date=datetime(2025,1,1),
    schedule_interval="@daily",
    catchup=False,
    default_args={"retries": 1, "retry_delay": timedelta(minutes=5)}
) as dag:

    create_cluster = DataprocCreateClusterOperator(
        task_id="create_cluster",
        project_id=PROJECT_ID,
        cluster_name=CLUSTER_NAME,
        region=REGION,
        cluster_config={"master_config": {"num_instances": 1}}
    )

    submit_spark = DataprocSubmitPySparkJobOperator(
        task_id="submit_spark",
        project_id=PROJECT_ID,
        region=REGION,
        main=PYSPARK_URI,
        cluster_name=CLUSTER_NAME
    )

    submit_hive = DataprocSubmitJobOperator(
        task_id="submit_hive",
        project_id=PROJECT_ID,
        region=REGION,
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
    job={
        "placement": {"cluster_name": CLUSTER_NAME},
        "hiveJob": {
            "queryFileUri": HIVE_SCRIPT_GCS,
            # or "queryList": {"queries": ["INSERT ...", "ANOTHER ..."]}
        }
    }
)

delete_cluster = DataprocDeleteClusterOperator(
    task_id="delete_cluster",
    project_id=PROJECT_ID,
    cluster_name=CLUSTER_NAME,
    region=REGION,
    trigger_rule="all_done"
)

create_cluster >> submit_spark >> submit_hive >> delete_cluster
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

15. A customer wants to reduce costs by using ephemeral Dataproc clusters. How would you implement this in Composer?

Step 1. Adopt a create-submit-delete pattern in the DAG: create an ephemeral cluster per run, submit job(s), then delete the cluster with trigger_rule="all_done".

Step 2. Use preemptible worker nodes (num_preemptible_instances) to reduce cost for batch workloads that tolerate interruptions.

Step 3. Right-size machine types and worker counts, and consider autoscaling policies to scale down quickly when load is low.

Step 4. Use Dataproc Workflow Templates with per-run instantiation if you want a managed, repeatable way to create ephemeral clusters and run multiple jobs in sequence.

Step 5. Cache or stage shared resources in GCS (jars, scripts, datasets) so clusters remain light and startup time is minimized.

Step 6. Use shorter image versions and initialization actions tuned for speed to reduce cluster startup latency and therefore cost.

Step 7. Limit maximum runtime and add cluster deletion timeouts/guards to avoid “zombie” clusters (e.g., use monitoring or a periodic cleanup DAG to remove orphaned clusters).

Step 8. Track and emit cost/usage metrics per run (cluster lifetime, vCPU-hours, bytes processed) so you can optimize further and prove cost savings.

Step 9. If many small jobs exist, consider batching small jobs into a single ephemeral cluster run to amortize startup cost, or use lightweight serverless products when appropriate.

Step 10. Enforce policies (via templates or IaC) so ephemeral cluster creation follows cost-saving defaults (preemptible workers, small masters, autoscaling) and developers cannot spin up expensive clusters ad-hoc.

Example: create-run-delete pattern with preemptible workers using DataprocCreateClusterOperator and DataprocDeleteClusterOperator:

```
from airflow import DAG

from airflow.providers.google.cloud.operators.dataproc import
DataprocCreateClusterOperator, DataprocDeleteClusterOperator,
DataprocSubmitPySparkJobOperator

from datetime import datetime, timedelta


CLUSTER_NAME = "ephemeral-cost-{{ ds_nodash }}"

PROJECT_ID = "my-gcp-project"

REGION = "us-central1"

PYSPARK_URI = "gs://my-bucket/jobs/job.py"
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
with DAG("dataproc_ephemeral_cost_savings", start_date=datetime(2025,1,1),
schedule_interval="@daily", catchup=False,
        default_args={"retries": 1, "retry_delay": timedelta(minutes=5)}) as dag:

    create_cluster = DataprocCreateClusterOperator(
        task_id="create_cluster",
        project_id=PROJECT_ID,
        cluster_name=CLUSTER_NAME,
        region=REGION,
        cluster_config={
            "master_config": {"num_instances": 1, "machine_type_uri": "n1-standard-4"},
            "worker_config": {
                "num_instances": 2,
                "num_preemptible_instances": 4,  # cost-saving
                "machine_type_uri": "n1-standard-4",
            },
            # consider autoscalingConfig or secondary worker pools
        },
    )

    submit_job = DataprocSubmitPySparkJobOperator(
        task_id="submit_job",
        project_id=PROJECT_ID,
        region=REGION,
        main=PYSPARK_URI,
        cluster_name=CLUSTER_NAME
    )

    delete_cluster = DataprocDeleteClusterOperator(
        task_id="delete_cluster",
        project_id=PROJECT_ID,
        cluster_name=CLUSTER_NAME,
        region=REGION,
        trigger_rule="all_done",
    )
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
create_cluster >> submit_job >> delete_cluster
```

16. A DAG must start whenever a new file lands in a GCS bucket. How would you configure this?

Step 1. Configure GCS notifications to publish object-change events to a Pub/Sub topic for the target bucket (GCS -> Pub/Sub) so events are pushable and reliable.

Step 2. Create a lightweight Cloud Function (or Cloud Run) that is triggered by that Pub/Sub topic, filters relevant events (prefix/file type), and then triggers the Composer DAG. Pass file metadata (bucket, name, eventTime) in the payload.

Step 3. Trigger the DAG from the function using the Airflow REST API (experimental or stable REST endpoint) or via the Composer REST trigger (TriggerDagRun API) with dag_run.conf carrying the file path; authenticate using a service account with least privilege.

Step 4. Alternatively, if you prefer pure-in-Airflow handling, deploy a small continuously-running Pub/Sub subscriber task in Composer that uses a **PubSubPullSensor** or a custom operator to pull messages and create DAG runs (but prefer Cloud Function to avoid long-running tasks in workers).

Step 5. In the DAG, make the first task idempotent and validate the file (checksum, filename pattern) before processing so duplicate notifications don't cause duplicate work.

Step 6. Use per-file run_id or include file fingerprint in dag_run.conf so retries and replayed events can be detected and de-duped.

Step 7. Secure the trigger path: restrict which service account can call the Composer/airflow REST endpoint, validate incoming payloads, and rotate keys/secrets via Secret Manager.

Step 8. Add quick pre-checks in the DAG (file size, file age) to avoid processing partial/temporary uploads; optionally use object metadata or a small "done" marker file strategy.

Step 9. Monitor Pub/Sub dead-lettering and Cloud Function errors; surface failures that prevent DAG triggering and set alerts for missed events.

Step 10. Test end-to-end by uploading test files, verifying Pub/Sub events, Cloud Function invocation, DAG run creation, and successful idempotent processing.

Cloud Function -> TriggerDagRun example (Python snippet):

```
# Cloud Function (python) triggered by Pub/Sub message (GCS notification)

import base64

import json

import google.auth.transport.requests

import google.oauth2.id_token

import requests

import os
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
AIRFLOW_TRIGGER_URL = "https://<composer-airflow-endpoint>/api/v1/dags/<dag_id>/dagRuns"

SERVICE_ACCOUNT_EMAIL = "composer-trigger-sa@my-project.iam.gserviceaccount.com"


def pubsub_handler(event, context):

    data = json.loads(base64.b64decode(event['data']).decode())

    bucket = data['bucket']

    name = data['name']

    payload = {"conf": {"bucket": bucket, "name": name}}


    # Acquire ID token for authentication to Airflow endpoint (if using IAP or Cloud Run style)

    auth_req = google.auth.transport.requests.Request()

    token = google.oauth2.id_token.fetch_id_token(auth_req, AIRFLOW_TRIGGER_URL)


    headers = {"Authorization": f"Bearer {token}", "Content-Type": "application/json"}

    resp = requests.post(AIRFLOW_TRIGGER_URL, json=payload, headers=headers)

    resp.raise_for_status()
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

17. You need to archive files after processing them. How would you design your Composer workflow?

Step 1. Define a clear post-processing stage in the DAG that runs only after successful processing (use default trigger rules so archive runs only on success).

Step 2. Implement the archive step to move the file to a separate GCS prefix/bucket (e.g., gs://bucket/processed/yyyy-mm-dd/) using a **GcsToGcsOperator** or a simple PythonOperator with the GCS Hook.

Step 3. Make the archive operation atomic from the pipeline's perspective: copy then verify then delete the original; or use GCS rename (copy+delete) and validate destination exists before deleting source.

Step 4. Add metadata when archiving (set custom metadata or add a manifest/log entry) with processing timestamps, job id, and checksum for auditing and future replay.

Step 5. Handle failures safely: set archive operator to retry on transient errors and set its trigger_rule to "all_done" only if you still want to archive failed inputs; otherwise keep it "all_success".

Step 6. For large volumes, batch archive moves to reduce API calls (e.g., archive many files with a single operator or a parallel map with bounded concurrency using pools).

Step 7. Retention policy: combine archiving with lifecycle rules on the archive bucket (auto-delete or transition to cold storage) and record retention policy in docs.

Step 8. Keep idempotency: if archive task re-runs, detect that file is already archived (by checking destination path or manifest) and skip instead of duplicating or erroring.

Step 9. Emit archive events/metrics (size moved, number of files) and alert on archive failures so manual cleanup doesn't accumulate.

Step 10. Provide an archive verification job (periodic) that samples archived files and verifies checksums and existence to ensure long-term integrity.

GCS archive step example using GCSToGCSOperator:

```
from airflow.providers.google.cloud.transfers.gcs_to_gcs import GCSToGCSOperator

archive = GCSToGCSOperator(
    task_id="archive_files",
    source_bucket="my-bucket",
    source_object="incoming/{{ ds }}/{{ ti.xcom_pull(key='file_name') }}",
    destination_bucket="my-bucket-archive",
    destination_object="processed/{{ ds }}/",
    move_object=True  # copy then delete original
)
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

18. Multiple file formats (CSV, JSON, Parquet) are landing in the same GCS bucket. How would you orchestrate ingestion?

Step 1. Implement an initial detection step that lists new files and classifies them by format (use file extension or content sniffing) so each file is routed to the correct parser.

Step 2. Use a single event-driven trigger (GCS -> Pub/Sub -> Cloud Function -> trigger DAG) that passes file metadata; let the DAG decide format-specific processing rather than having separate buckets.

Step 3. Create small format-specific processing sub-DAGs or TaskGroups (CSV branch, JSON branch, Parquet branch) that contain parsing, validation, and schema normalization logic.

Step 4. Normalize outputs to a common canonical schema (use a transform step that maps CSV/JSON/Parquet fields to canonical columns) or write raw data into a staging zone preserving original format and then a transform job to canonicalize.

Step 5. Use dynamic task generation so each file gets an isolated processing task (or mapped tasks) that runs the correct operator/logic based on file type.

Step 6. For CSV/JSON, use **GCSToBigQueryOperator** (with schema or autodetect) or a PythonOperator with pandas/fastparquet for pre-processing; for Parquet prefer direct BigQuery load or BigQueryExternalTable if schema compat is known.

Step 7. Ensure idempotency per file (use file fingerprinting, write marker files on success, or persist processed file names to a datastore) to avoid double-processing on duplicate events.

Step 8. Maintain schema validation and transformation rules centrally (YAML/JSON schema registry) so all format handlers apply the same mapping and type casts.

Step 9. Parallelize safely: use pools or concurrency limits to protect downstream systems (BigQuery quotas) and prioritize critical formats if needed.

Step 10. Monitor format-specific error rates and keep sample raw files for debugging; add automated alerts for new/unexpected formats so you can extend handlers quickly.

Example: branching by file extension with a BranchPythonOperator and format handlers (CSV, JSON, Parquet) — handler names bolded below:

BranchPythonOperator to choose format (code snippet):

```
from airflow import DAG

from airflow.operators.python import BranchPythonOperator, PythonOperator

from airflow.operators.empty import EmptyOperator

from datetime import datetime


def choose_handler(**kwargs):

    file_name = kwargs['dag_run'].conf.get('name')  # passed from trigger

    if file_name.endswith('.csv'):

        return 'handle_csv'

    if file_name.endswith('.json'):
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
        return 'handle_json'
    if file_name.endswith('.parquet'):
        return 'handle_parquet'
    return 'handle_unknown'

def handle_csv(**kwargs):
    # CSV processing logic: validation, convert, load to BQ
    pass

def handle_json(**kwargs):
    # JSON processing logic
    pass

def handle_parquet(**kwargs):
    # Parquet processing logic
    pass

with DAG("multi_format_ingest", start_date=datetime(2025,1,1), schedule_interval=None,
catchup=False) as dag:
    choose = BranchPythonOperator(task_id="choose_format",
python_callable=choose_handler, provide_context=True)

    csv_task = PythonOperator(task_id="handle_csv", python_callable=handle_csv)
    json_task = PythonOperator(task_id="handle_json", python_callable=handle_json)
    parquet_task = PythonOperator(task_id="handle_parquet", python_callable=handle_parquet)
    unknown = EmptyOperator(task_id="handle_unknown")
    join = EmptyOperator(task_id="join", trigger_rule="one_success")

    choose >> [csv_task, json_task, parquet_task, unknown] >> join
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

19. A streaming pipeline ingests data via Pub/Sub. You need to trigger batch jobs daily to aggregate this data. How would you design it?

Step 1. Keep streaming ingestion decoupled from batch aggregation: let Pub/Sub → Dataflow (streaming) ingest raw events into a durable storage (BigQuery raw table, Bigtable, or GCS parquet files) so the batch job reads a stable snapshot.

Step 2. Choose the batch aggregation store and partitioning strategy (partitioned BigQuery tables by ingestion date or hourly files in GCS) to make daily reads efficient.

Step 3. Schedule a daily Composer DAG that runs after the day's events are expected to be landed (use time-based schedule and/or a light readiness check like presence of a daily manifest or minimum row count).

Step 4. In the DAG, include a data-availability check (BigQuery count query or GCS prefix existence) to avoid running empty aggregations; branch/short-circuit if no data.

Step 5. Use a batch job operator appropriate for aggregation: BigQueryInsertJobOperator for SQL aggregations, or DataflowStartFlexTemplateOperator / DataflowStartTemplateOperator if the aggregation logic needs custom Beam code.

Step 6. Make batch aggregation idempotent and partition-aware: write results to a partitioned summary table with WRITE_TRUNCATE on the target partition or use MERGE to update only that day's partition.

Step 7. Use XComs or structured logs to capture job metadata (rows processed, runtime, job_id) and persist that to a monitoring table for auditing and SLA checks.

Step 8. Add retries/backoff for transient failures and sensible timeouts; ensure retries don't duplicate results by using idempotent writes or detecting and skipping already-processed partitions.

Step 9. Add alerting/SLAs on the DAG so operators are notified if the daily aggregation fails or is delayed beyond the SLA window.

Step 10. Optionally, provide a manual re-run path (parameterized DAG run with date) so operators can re-run a single day's aggregation without touching other partitions.

BigQuery daily aggregation (example):

```
from airflow import DAG

from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator,
BigQueryCheckOperator

from datetime import datetime


with DAG("daily_stream_aggregation", start_date=datetime(2025,1,1), schedule_interval="0 2 *
* *", catchup=False) as dag:

    # Example: aggregate streaming raw table into daily_summary partition

    agg = BigQueryInsertJobOperator(

        task_id="aggregate_to_daily_summary",
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
    configuration={
      "query": {
        "query": """
        CREATE OR REPLACE TABLE `myproj.analytics.daily_summary`
        PARTITION BY DATE(event_date) AS
        SELECT
          DATE(event_timestamp) AS event_date,
          user_id,
          COUNT(*) AS events_count,
          SUM(value) AS total_value
        FROM `myproj.streaming.raw_events`
        WHERE DATE(event_timestamp) = '{{ ds }}'
        GROUP BY event_date, user_id
        """,
        "useLegacySql": False
      }
    }
  )

  check = BigQueryCheckOperator(
    task_id="check_rows",
    sql="SELECT COUNT(1) FROM `myproj.analytics.daily_summary` WHERE event_date = '{{
ds }}'",
    use_legacy_sql=False
  )

  agg >> check
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

20. Some Pub/Sub messages are failing processing. How would you handle them in Composer?

Step 1. Surface failures early: ensure your downstream consumer (Dataflow or Composer task) logs message-level errors and writes failing message payloads and metadata to a dead-letter location (DLQ) — e.g., Pub/Sub Dead Letter Topic or GCS/BigQuery table.

Step 2. For Composer-driven processing, wrap per-message processing in idempotent logic and catch/record exceptions rather than crashing the whole batch; record the failed message and failure reason.

Step 3. Implement a dead-letter processing DAG that periodically reads the DLQ (Pub/Sub dead-letter subscription, GCS prefix, or BigQuery table) and attempts retries with backoff or enrichment/repair steps.

Step 4. Provide triage metadata: include original message, timestamp, error stack, and context (run_id, task_id) so engineers can reproduce and fix schema/transform issues.

Step 5. For transient errors (rate limits, downstream service), implement automatic retry logic in the DLQ DAG with a controlled number of attempts; escalate to humans after thresholds are exceeded.

Step 6. For poison-message scenarios (malformed or incompatible), implement a quarantine flow: mark message as permanently failed after N attempts and notify stakeholders to investigate and fix the upstream producer.

Step 7. Use schema validation upfront (JSON schema, Protobuf, or AVRO) in the streaming pipeline to fail-fast and push invalid messages to DLQ rather than allow downstream corruption.

Step 8. Expose a replay mechanism: keep raw messages (or pointers) so once a fix is in place you can reprocess the quarantined messages via a Composer DAG that reads DLQ and replays to main ingestion (or to a repair job).

Step 9. Monitor DLQ size and failure patterns with dashboards and alerts (increase in DLQ rate, recurring error types) to detect regressions.

Step 10. Secure the DLQ and access to replay tools so only authorized operators can replay or delete problematic messages.

DLQ replay DAG example (reads failed messages from BigQuery DLQ and retries):

```
from airflow import DAG

from airflow.operators.python import PythonOperator

from airflow.providers.google.cloud.hooks.pubsub import PubSubHook

from datetime import datetime, timedelta

from google.cloud import bigquery
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
def fetch_and_republish(**kwargs):
  bq = bigquery.Client()
  query = """
    SELECT message_data, failure_reason
    FROM `myproj.dlq.failed_messages`
    WHERE processed = FALSE
    LIMIT 100
  """
  rows = bq.query(query).result()
  pub = PubSubHook(project_id="myproj")
  for row in rows:
    pub.publish(project="myproj", topic="raw-events", messages=[row.message_data])
    # mark as processed in BQ (omitted here for brevity)


with DAG("dlq_replay", start_date=datetime(2025,1,1), schedule_interval="*/15 * * * *",
catchup=False,
     default_args={"retries": 0}) as dag:
  replay = PythonOperator(task_id="fetch_and_republish",
python_callable=fetch_and_republish)
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

21. You must coordinate a Pub/Sub → Dataflow → BigQuery pipeline using Composer. How would you orchestrate it?

Step 1. Keep Pub/Sub ingestion and Dataflow streaming job(s) independent of Composer for continuous ingestion — Dataflow streaming job subscribes to Pub/Sub and writes raw events to BigQuery or GCS.

Step 2. Use Composer to manage lifecycle and batch operations: start/stop Dataflow jobs (if you prefer ephemeral streaming jobs), perform backfill runs, and run scheduled batch aggregations on BigQuery.

Step 3. If you start Dataflow jobs from Composer, use DataflowStartTemplateOperator/DataflowStartFlexTemplateOperator to launch streaming jobs and capture job_id via XCom for later monitoring or teardown.

Step 4. Monitor Dataflow job status with DataflowJobStatusSensor or custom polling via DataflowHook; have Composer react to job failures (alert, restart, or run a drain/cleanup flow).

Step 5. Ensure the Dataflow pipeline writes durable intermediate artifacts (partitioned BigQuery tables or daily files in GCS) that Composer batch jobs can safely read for downstream processing.

Step 6. Use Composer to run batch steps after Dataflow output is finalized (e.g., nightly summary SQL in BigQuery using BigQueryInsertJobOperator), or to trigger downstream transformations that depend on the streaming output.

Step 7. Implement exactly-once/duplication controls between Dataflow and BigQuery (use dedup keys, write with MERGE, or use BigQuery streaming insert de-duplication patterns) so Composer jobs can safely reprocess if needed.

Step 8. For error handling, have Dataflow push critical dead-letter events to Pub/Sub/GCS and let Composer run remediation/replay DAGs to handle reprocessing of failed messages.

Step 9. Use IAM and least-privilege service accounts: Composer service account can start/monitor Dataflow, and Dataflow worker service account writes to BigQuery/GCS with minimal needed roles.

Step 10. Add observability across the flow: correlate Pub/Sub message timestamps, Dataflow job_id, and BigQuery load job_ids in logs/monitoring so debugging and SLA reporting across components is straightforward.

Example: Composer DAG that starts a streaming Dataflow job, monitors it, then runs a nightly BigQuery aggregation
(Dataflow start + DataflowJobStatusSensor + BigQueryInsertJobOperator example):

```
from airflow import DAG

from airflow.providers.google.cloud.operators.dataflow import
DataflowStartFlexTemplateOperator

from airflow.providers.google.cloud.sensors.dataflow import DataflowJobStatusSensor

from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator

from datetime import datetime
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
with DAG("pubsub_dataflow_bigquery", start_date=datetime(2025,1,1),
schedule_interval="@daily", catchup=False) as dag:


    start_stream = DataflowStartFlexTemplateOperator(
        task_id="start_streaming_dataflow",
        project_id="myproj",
        region="us-central1",
        body={
            "launchParameter": {
                "jobName": "streaming-job-{{ ds_nodash }}",
                "containerSpecGcsPath": "gs://my-templates/streaming-flex.json",
                "parameters": {"inputSubscription": "projects/myproj/subscriptions/raw-sub"}
            }
        }
    )
    monitor_stream = DataflowJobStatusSensor(
        task_id="monitor_stream_job",
        project_id="myproj",
        region="us-central1",
        job_id="{{ task_instance.xcom_pull('start_streaming_dataflow', key='job_id') }}",
        poke_interval=300,
        timeout=60 * 60 * 24  # optional
    )
    nightly_agg = BigQueryInsertJobOperator(
        task_id="nightly_aggregate",
        configuration={
            "query": {
                "query": "SQL_TO_AGGREGATE_FROM_raw_table WHERE DATE(event_timestamp) = '{{
ds }}'",
                "useLegacySql": False
            }
        },
        trigger_rule="all_success"
    )


    start_stream >> monitor_stream >> nightly_agg
```

© Shubham Wadekar

22. You must design a multi-step pipeline: GCS → Dataflow → BigQuery. How would you orchestrate it in Composer?

Step 1. Define a clear contract: what Dataflow expects from GCS (file pattern, schema) and what BigQuery table/schema will receive the output.

Step 2. Trigger the DAG either on a schedule or on file arrival (Pub/Sub trigger) and pass the specific GCS URIs into the DAG run via dag_run.conf.

Step 3. Add a lightweight pre-check task that validates file existence, size, and basic schema (sample a few records) to avoid launching expensive jobs for bad inputs.

Step 4. Start one or more Dataflow jobs using a template operator (DataflowStartTemplateOperator / DataflowStartFlexTemplateOperator) and pass the GCS input paths and output staging paths as parameters.

Step 5. Capture the Dataflow job_id in XCom so downstream tasks can monitor and reference it.

Step 6. Monitor Dataflow to completion using a Dataflow job status sensor or a polling task that calls the Dataflow API; fail fast on irrecoverable errors and gather diagnostics on failures.

Step 7. Write Dataflow outputs to a durable staging location (partitioned GCS or a BigQuery staging table). Prefer partitioned, discoverable locations so downstream loads are simple and idempotent.

Step 8. Load or transform staged output into the final BigQuery table using BigQuery operators: either a direct BigQuery load (if Dataflow wrote files) or a BigQuery SQL MERGE/INSERT if Dataflow wrote to a staging table.

Step 9. Use idempotent load patterns: unique run identifiers, partitioned writes, and MERGE statements to avoid duplicates when retries occur.

Step 10. Add post-load validation, cleanup (archive or delete staging files), logging of job metrics (job_id, rows processed, run_time), and alerts/SLAs for operational visibility.

Code (example DAG: start Dataflow flex template, monitor it, then run a BigQuery load). The operator names below are bolded per your preference: **DataflowStartFlexTemplateOperator**, **DataflowJobStatusSensor**, **BigQueryInsertJobOperator**.

```
from airflow import DAG

from airflow.providers.google.cloud.operators.dataflow import
DataflowStartFlexTemplateOperator

from airflow.providers.google.cloud.sensors.dataflow import DataflowJobStatusSensor

from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator

from airflow.operators.empty import EmptyOperator

from datetime import datetime
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
with DAG("gcs_dataflow_bq", start_date=datetime(2025,1,1), schedule_interval="@daily",
catchup=False) as dag:


  start_dataflow = DataflowStartFlexTemplateOperator(
    task_id="start_dataflow",
    project_id="my-project",
    region="us-central1",
    body={
      "launchParameter": {
        "jobName": "etl-{{ ds_nodash }}",
        "containerSpecGcsPath": "gs://my-templates/flex-template.json",
        "parameters": {
          "input": "gs://my-bucket/inputs/{{ ds }}/",
          "output": "gs://my-bucket/staging/{{ ds }}/"
        }
      }
    }
  )


  monitor_dataflow = DataflowJobStatusSensor(
    task_id="monitor_dataflow",
    project_id="my-project",
    region="us-central1",
    job_id="{{ task_instance.xcom_pull(task_ids='start_dataflow', key='job_id') }}",
    poke_interval=60,
    timeout=60 * 60 * 4
  )

  load_bq = BigQueryInsertJobOperator(
    task_id="load_to_bq",
    configuration={
      "query": {
        "query": """
        -- example: load from staged files into partitioned table for {{ ds }}
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
        CREATE OR REPLACE TABLE `myproj.analytics.daily_agg`
        PARTITION BY DATE(run_date) AS
        SELECT
          run_date,
          key,
          SUM(value) AS total_value
        FROM `myproj.staging.etl_output_{{ ds_nodash }}`
        GROUP BY run_date, key
        """,
        "useLegacySql": False
      }
    }
)

cleanup = EmptyOperator(task_id="cleanup")  # replace with GCS delete operator if needed

start_dataflow >> monitor_dataflow >> load_bq >> cleanup
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

23. A business requires incremental loads into BigQuery. How would you design an ELT DAG?

Step 1. Define the incremental key (e.g., event_timestamp, last_modified_ts, or an increasing ID) and choose whether source-driven watermarking or a change-log approach will be used.

Step 2. Implement a reliable source-read step that extracts only new/changed rows since the last successful run. Persist the watermark (last_processed_timestamp or last_id) in a durable store (BigQuery table, Cloud SQL, or Airflow Variable).

Step 3. Stage incremental data into a staging area: either a staging BigQuery table or files on GCS partitioned by run_date/run_id for traceability.

Step 4. Validate and normalize staged data (schema checks, type casts, deduplication) so downstream MERGE logic won't fail.

Step 5. Merge staged incremental data into the target BigQuery table using a MERGE statement that handles INSERTS, UPDATES, and (optionally) DELETES based on CDC semantics.

Step 6. Use transactions or atomic SQL patterns where possible (e.g., CREATE OR REPLACE TABLE for new partitions, or MERGE for targeted updates) to avoid partial state.

Step 7. After successful MERGE, update the persisted watermark atomically so the next run picks up from the right point; record audit metadata (rows processed, errors, run_id).

Step 8. Add idempotency protections (use unique keys and dedupe logic) so retries do not cause duplicated records.

Step 9. Emit metrics and SLA alerts (eg. low/high row counts, long run times, merge conflicts) and provide a manual re-run interface that can take a from/to watermark.

Step 10. Add tests and a staging environment to validate MERGE logic against sample incremental batches before rolling into production.

Code (example ELT DAG showing extract -> stage -> MERGE). The MERGE operator below uses **BigQueryInsertJobOperator** for the MERGE query.

```
from airflow import DAG

from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator

from airflow.operators.python import PythonOperator

from datetime import datetime


def get_last_watermark(**kwargs):

    # read watermark from a small metadata table or Airflow Variable

    # for demo return yesterday's date

    return kwargs.get('ds')
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
def extract_incremental(**kwargs):
    last_watermark = kwargs['ti'].xcom_pull(task_ids='get_watermark')
    # extract records changed since last_watermark and write to myproj.staging.inc_{{ ds_nodash }}
    # This is a placeholder for actual extraction logic.

with DAG("incremental_elt", start_date=datetime(2025,1,1), schedule_interval="@daily", catchup=False) as dag:

    get_watermark = PythonOperator(task_id="get_watermark", python_callable=get_last_watermark, provide_context=True)

    extract = PythonOperator(task_id="extract_incremental", python_callable=extract_incremental, provide_context=True)

    merge_to_prod = BigQueryInsertJobOperator(
        task_id="merge_to_prod",
        configuration={
            "query": {
                "query": """
                MERGE `myproj.analytics.target_table` T
                USING `myproj.staging.inc_{{ ds_nodash }}` S
                ON T.id = S.id
                WHEN MATCHED THEN UPDATE SET
                  col1 = S.col1,
                  col2 = S.col2,
                  last_modified = S.last_modified
                WHEN NOT MATCHED THEN
                  INSERT (id, col1, col2, last_modified) VALUES (S.id, S.col1, S.col2, S.last_modified)
                """,
                "useLegacySql": False
            }
        }
    )
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
record_watermark = PythonOperator(task_id="record_watermark", python_callable=lambda **kw: print("record watermark"))


get_watermark >> extract >> merge_to_prod >> record_watermark
```

24. A transformation step fails, but the business still wants the load step to run with partial data. How would you handle this?

Step 1. Design the pipeline to separate transformation and load into distinct stages with clear artifacts (staging outputs) so the load can consume whatever validated output exists.

Step 2. Make the transformation step produce partial outputs and a manifest that indicates which partitions/files succeeded and which failed.

Step 3. When a transform task fails, capture failure metadata (failed partitions, error details) and let the DAG proceed along a controlled path rather than failing the whole run.

Step 4. Use a Branch or ShortCircuit step that decides whether to proceed to load with partial data based on thresholds (minimum rows/files processed) or explicit business rules.

Step 5. In the load step, use MERGE/append behavior that tolerates missing partitions (write only the available partitions) and ensure idempotency so partial loads are safe to retry later after reprocessing missing parts.

Step 6. Mark the DAG run with warnings (log, metrics, and an alert) so stakeholders are aware that the run completed with partial data and what data is missing.

Step 7. Create a compensating workflow to re-run only the failed transform partitions and then re-run the load for those partitions, or provide a manual re-run path.

Step 8. If partial loads are acceptable only for non-critical consumers, use tags/metadata to indicate data quality; let critical consumers block until full data is available.

Step 9. Persist partial-run artifacts (manifest, failed file list) to GCS/BigQuery so operators can inspect and take corrective action.

Step 10. Add tests and SLAs that define acceptable partial-load thresholds and automate escalation if repeated partial runs occur.

Code (example pattern: transform may fail but load proceeds when minimum data exists). The load operator below uses **BigQueryInsertJobOperator**; branching uses **BranchPythonOperator**:

```
from airflow import DAG

from airflow.operators.python import PythonOperator, BranchPythonOperator

from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator

from airflow.operators.empty import EmptyOperator

from datetime import datetime
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
def transform(**kwargs):
    # attempt transforms; write successful partitions to gs://.../staging/{{ ds }}/
    # return list of successful partitions or empty list if none
    return ["part=1", "part=2"]  # example

def decide_proceed(**kwargs):
    success_parts = kwargs['ti'].xcom_pull(task_ids='transform')
    # business rule: proceed if at least one partition succeeded
    if success_parts and len(success_parts) >= 1:
        return 'load_partial'
    else:
        return 'abort_load'

def record_warning(**kwargs):
    print("Transform failed partially; proceeding with partial load. See manifest at gs://...")

with DAG("partial_transform_then_load", start_date=datetime(2025,1,1),
schedule_interval="@daily", catchup=False) as dag:

    transform_task = PythonOperator(task_id="transform", python_callable=transform)

    decide = BranchPythonOperator(task_id="decide", python_callable=decide_proceed,
provide_context=True)

    load_partial = BigQueryInsertJobOperator(
        task_id="load_partial",
        configuration={
            "query": {
                "query": """
                -- load whatever is present in staging for this run
                INSERT INTO `myproj.analytics.target_table` (cols...)
                SELECT * FROM `myproj.staging.partial_{{ ds_nodash }}`
                """,
                "useLegacySql": False
            }
        }
```

© Shubham Wadekar

```
)

abort = EmptyOperator(task_id="abort_load")
warn = PythonOperator(task_id="record_warning", python_callable=record_warning)

transform_task >> decide
decide >> [load_partial, abort]
load_partial >> warn
```

25. A DAG must run every hour, but if it misses a run, it should catch up later. How would you configure this?

Step 1. Set the DAG schedule to run hourly (cron 0 * * * * or @hourly) so Airflow schedules one execution per hour.

Step 2. Enable catchup by leaving catchup=True (the default) so missed execution_dates are queued and run when the scheduler next has capacity.

Step 3. Choose an appropriate start_date in the past (not datetime.now() at runtime) so Airflow knows the historical window to backfill from.

Step 4. Limit concurrency with max_active_runs on the DAG so catch-up runs don't overwhelm workers — pick a small number (e.g., 3) to throttle parallel catch-up execution.

Step 5. Avoid depends_on_past=True for most tasks unless absolutely needed, because it can block catch-up if one run is stuck; prefer depends_on_past=False.

Step 6. Use idempotent tasks and partitioned outputs (run_id / ds) so re-running missed runs is safe and won't create duplicates.

Step 7. Add sensible timeouts and retries to individual tasks so slow jobs don't indefinitely delay catch-up progress.

Step 8. Monitor progress: log catch-up progress (counts of queued vs finished runs) and alert if catch-up backlog grows beyond an SLA.

Step 9. If very large backfills are possible, consider a manual backfill path (airflow dags backfill or a controlled re-run) instead of automatic catchup for extreme volumes.

Step 10. Test the behavior in staging by simulating missed schedules and verifying runs are executed later according to max_active_runs and resource constraints.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Hourly DAG with catchup enabled (example):

```
from airflow import DAG

from airflow.operators.empty import EmptyOperator

from datetime import datetime, timedelta


with DAG(

    dag_id="hourly_with_catchup",

    start_date=datetime(2025, 1, 1),    # fixed past start_date

    schedule_interval="@hourly",

    catchup=True,                # enable automatic catchup

    max_active_runs=3,           # throttle concurrent catchup runs

    default_args={"depends_on_past": False},

) as dag:

    start = EmptyOperator(task_id="start")
```

26. A data pipeline must complete within 30 minutes, or an SLA alert should be triggered. How would you set this up?

Step 1. Define the SLA target as a 30-minute limit and decide whether it applies to the full DAG or to a specific terminal task that represents pipeline completion.

Step 2. Use Airflow task-level SLA by setting the sla argument on the task(s) you want monitored (e.g., sla=timedelta(minutes=30)).

Step 3. Implement an sla_miss_callback at the DAG level (or set sla_miss_callback in default_args) that sends alerts (Slack, email, pager) when an SLA is missed.

Step 4. Ensure tasks have reasonable execution_timeout and retries so long-running tasks are bounded and don't silently exceed SLA windows.

Step 5. Optionally add a final "completion" task (EmptyOperator) whose SLA is the 30-minute window; this simplifies SLA monitoring to a single task.

Step 6. Instrument metrics (task durations, percentiles) and dashboards so you can proactively tune tasks to meet the 30-minute SLA.

Step 7. If retries could push completion past SLA, decide whether SLA should consider retries or only first-attempt latency — document and code accordingly.

Step 8. Test the SLA handling in staging by simulating slow tasks and ensuring the sla_miss_callback fires and notifications are delivered.

Step 9. For strict SLAs, consider running critical tasks with higher priority or dedicated pools to reduce resource contention.

Step 10. Maintain run-level metadata (start_time, end_time, duration) in a monitoring store so SLA history and trends are visible.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

SLA on tasks with an SLA miss callback (example):

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.empty import EmptyOperator
from datetime import datetime, timedelta

def on_sla_miss(dag, task_list, blocking_task_list, slas, blocking_tis):
    # send alert to Slack/email with details
    print("SLA missed for DAG:", dag.dag_id, "tasks:", task_list)

def do_work():
    # long-running work
    pass

with DAG(
    dag_id="pipeline_with_sla",
    start_date=datetime(2025,1,1),
    schedule_interval="@daily",
    sla_miss_callback=on_sla_miss,  # DAG-level SLA miss handler
    catchup=False,
) as dag:

    work = PythonOperator(
        task_id="etl_work",
        python_callable=do_work,
        sla=timedelta(minutes=30),          # task-level SLA = 30 minutes
        execution_timeout=timedelta(minutes=45)  # hard bound on single task
    )

    done = EmptyOperator(task_id="done", sla=timedelta(minutes=30))
    work >> done
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

27. A weekend job should run only on Saturdays. How would you configure scheduling?

Step 1. Choose a cron expression that fires only on Saturday; for midnight Saturday use 0 0 * * 6 (day-of-week 6 = Saturday).

Step 2. Alternatively use a cron with day name 0 0 * * SAT if your scheduler accepts names — prefer numeric 6 for portability.

Step 3. Set schedule_interval of the DAG to that cron and set start_date appropriately (a past date that aligns with the intended runs).

Step 4. Set catchup=False if you do not want historical backfills to run for missed Saturdays, or catchup=True if you want missed Saturdays to be replayed.

Step 5. If the job must run at a specific hour on Saturday (e.g., 03:30), adjust cron to 30 3 * * 6.

Step 6. Validate timezone: Airflow uses the DAG timezone (default UTC or set via tzinfo) — ensure the schedule maps to your business timezone (e.g., Asia/Kolkata).

Step 7. Add a clear DAG description mentioning it runs only on Saturdays so operators don't accidentally change schedule.

Step 8. Test in staging by setting the start_date and using airflow dags backfill or running with a test schedule_interval before switching to production cron.

Step 9. If business wants "only weekends" (both Saturday and Sunday), use 0 0 * * 6,0 or 0 0 * * SAT,SUN — but for only Saturday use 6.

Step 10. Monitor first few runs to confirm timing and timezone correctness.

DAG scheduled for Saturdays at 00:00 (example):

```
from airflow import DAG

from airflow.operators.empty import EmptyOperator

from datetime import datetime

import pendulum


local_tz = pendulum.timezone("Asia/Kolkata")


with DAG(

    dag_id="saturday_job",

    start_date=datetime(2025,1,4, tzinfo=local_tz),  # a Saturday in the past

    schedule_interval="0 0 * * 6",                # run only on Saturdays

    catchup=False,

) as dag:

    run = EmptyOperator(task_id="run")
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

28. A task connecting to an API fails due to intermittent network issues. How would you configure retries?

Step 1. Set per-task retries and retry_delay on the operator (e.g., retries=3, retry_delay=timedelta(seconds=30)), not globally, so only that API task gets the policy.

Step 2. Enable exponential backoff by using retry_exponential_backoff=True and set max_retry_delay so retries back off automatically and don't hammer the API.

Step 3. Use idempotency keys or upsert semantics in the API calls so retries don't create duplicates when the API eventually succeeds.

Step 4. For fine-grained retry control, implement retry logic inside the callable (tenacity or custom backoff) to inspect HTTP status codes and only retry on transient errors (5xx, connection errors, timeouts), not on 4xx client errors.

Step 5. Add jitter (randomized delay) to avoid thundering-herd retries across many tasks at the same time.

Step 6. Keep retries bounded (max attempts + max delay) and fail fast for persistent errors so you can alert and investigate rather than loop indefinitely.

Step 7. Use the Airflow on_failure_callback to capture diagnostics (request payload, headers, response body, stack trace) and persist them to GCS/BigQuery or send to an alerting channel for investigation.

Step 8. If the API has rate limits, combine retries with a throttling mechanism (queueing, token bucket, Cloud Tasks) to avoid hitting limits repeatedly.

Step 9. Prefer the provider HttpHook or SimpleHttpOperator for standardized behavior; when more control is needed, use a PythonOperator with a robust retry library.

Step 10. Test retries in staging by simulating transient failures (HTTP 502/503, timeouts) and verify backoff, jitter, idempotency, and alerts behave as expected.

PythonOperator with HttpHook and tenacity-style backoff (example):

```
from airflow import DAG

from airflow.operators.python import PythonOperator

from airflow.providers.http.hooks.http import HttpHook

from datetime import datetime, timedelta


# If tenacity is available in your environment, you can use it:

try:

    from tenacity import retry, wait_exponential, stop_after_attempt, retry_if_exception_type

except Exception:

    retry = None
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
def call_api_payload(payload):
    hook = HttpHook(method="POST", http_conn_id="my_api_conn")
    resp = hook.run(endpoint="/v1/process", json=payload, timeout=30)
    if not (200 <= resp.status_code < 300):
        resp.raise_for_status()
    return resp.json()


if retry:
    @retry(
        wait=wait_exponential(multiplier=2, min=4, max=60),
        stop=stop_after_attempt(5),
        retry=retry_if_exception_type(Exception)
    )
    def call_api_with_backoff(payload):
        return call_api_payload(payload)
else:
    # fallback simple backoff if tenacity not installed
    import time
    def call_api_with_backoff(payload):
        max_attempts = 5
        for attempt in range(1, max_attempts + 1):
            try:
                return call_api_payload(payload)
            except Exception as e:
                if attempt == max_attempts:
                    raise
                time.sleep(min(2 ** attempt, 60) + (attempt * 0.1))  # exponential + small jitter


def task_callable(**kwargs):
    payload = {"id": kwargs['ds_nodash']}
    result = call_api_with_backoff(payload)
    print("API result:", result)
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
with DAG("api_retry_policy", start_date=datetime(2025,1,1), schedule_interval=None, catchup=False) as dag:
  call_api = PythonOperator(
    task_id="call_api_with_backoff",
    python_callable=task_callable,
    retries=0  # retries handled inside callable
  )
```

29. A DAG fails at step 3 of 5. How would you design error handling so downstream tasks can still run?

Step 1. Classify tasks as critical vs non-critical so you know which failures must stop the flow and which can be tolerated or worked around.

Step 2. Decouple work into independent branches or TaskGroups; keep unrelated downstream tasks in separate branches so a failure in one branch doesn't block others.

Step 3. Use trigger_rule="all_done" (or one_success / none_failed_min_one_success) on downstream join tasks so they can run despite upstream failures, where business logic allows.

Step 4. Add a failure-handling task (runs on all_done) that records failure metadata (failed task id, error, run_id) to a manifest that downstream tasks can read to decide what to process.

Step 5. Use BranchPythonOperator or ShortCircuitOperator to decide at runtime whether to run a downstream task based on the presence/absence of required artifacts produced by step 3.

Step 6. Implement idempotent downstream tasks that can accept partial inputs (process only available partitions/files) so they can complete even when upstream is partially failed.

Step 7. For recoverable errors, push failed items to a retry queue (Pub/Sub/Cloud Tasks) by the failure capture task so they can be reprocessed independently without blocking the main flow.

Step 8. Use on_failure_callback to notify owners and optionally trigger a compensating DAG that remediates/fills the missing data later.

Step 9. Add clear logging and a manifest (success/failure per partition) so downstream tasks know exactly what succeeded and what is missing.

Step 10. Test this behavior with simulated partial failures to ensure downstream tasks run correctly and produce expected partial outputs and alerts.

Example: using trigger_rule all_done and failure recording (example):

```
from airflow import DAG
from airflow.operators.python import PythonOperator, BranchPythonOperator
from airflow.operators.empty import EmptyOperator
from airflow.utils.trigger_rule import TriggerRule
from datetime import datetime
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
def step1(): pass
def step2(): pass

def step3_may_fail(**kwargs):
    # simulate failure for demo; in reality run transformation and raise on fatal error
    raise Exception("step3 failed")

def record_failure(**kwargs):
    # record that step3 failed (write to GCS/BigQuery or XCom)
    ti = kwargs['ti']
    try:
        ti.xcom_pull(task_ids='step3')
    except Exception:
        ti.xcom_push(key='step3_failed', value=True)

def decide_continue(**kwargs):
    ti = kwargs['ti']
    failed = ti.xcom_pull(key='step3_failed', task_ids='record_failure')
    # business rule: proceed if we have at least partial data (for demo, proceed regardless)
    return 'downstream_task'

def downstream(**kwargs):
    # downstream should handle partial inputs
    print("running downstream (can handle partial data)")

with DAG("tolerate_step3_failure", start_date=datetime(2025,1,1), schedule_interval=None,
catchup=False) as dag:
    s1 = PythonOperator(task_id="step1", python_callable=step1)
    s2 = PythonOperator(task_id="step2", python_callable=step2)
    s3 = PythonOperator(task_id="step3", python_callable=step3_may_fail)

    # runs regardless of s3 success/failure
    rec = PythonOperator(task_id="record_failure", python_callable=record_failure,
trigger_rule=TriggerRule.ALL_DONE, provide_context=True)
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
branch = BranchPythonOperator(task_id="decide", python_callable=decide_continue, provide_context=True)

downstream_task = PythonOperator(task_id="downstream_task", python_callable=downstream)

end = EmptyOperator(task_id="end", trigger_rule=TriggerRule.ONE_SUCCESS)


s1 >> s2 >> s3 >> rec >> branch

branch >> [downstream_task] >> end
```

30. A business requires an alert whenever a DAG fails. How would you implement it?

Step 1. Use a DAG-level on_failure_callback so any task failure that causes the DAG run to fail triggers a single handler for notification (centralized and consistent).

Step 2. Implement the callback to capture context (dag_id, run_id, failed tasks, execution_date, error messages) and send to the chosen channel (Slack, email, PagerDuty, or Cloud Monitoring).

Step 3. Configure Airflow Connections / Variables to hold alerting credentials (Slack webhook URL, SMTP credentials, PagerDuty integration key) and never hardcode secrets in DAGs.

Step 4. For more advanced routing, filter the callback by DAG id or failure type and send to different channels or escalation paths accordingly.

Step 5. Optionally augment with task-level on_failure_callback for task-specific diagnostics (attach logs, stack traces, small excerpts) while keeping DAG-level callback for the primary alert.

Step 6. Integrate Airflow alerts with Cloud Monitoring (Stackdriver) or use a webhook that forwards to an incident management system so SLA-driven alerts are tracked and paged.

Step 7. Add retry/dedup logic in the notification function to avoid alert storms if many tasks fail simultaneously (group failures into one message when appropriate).

Step 8. Test alerting by injecting a controlled failure in staging and verifying the message format, recipients, and escalation path.

Step 9. Maintain a runbook link in the alert message with quick remediation steps, log links, and playbook instructions to reduce mean time to resolution.

Step 10. Log all alert events into a monitoring dataset (BigQuery/GCS) for auditing and trend analysis so you can reduce false positives over time.

DAG-level on_failure_callback (example):

```
from airflow import DAG

from airflow.operators.python import PythonOperator

from datetime import datetime, timedelta

import requests

import os
```

© Shubham Wadekar

```
# read Slack webhook from Airflow Variable/Connection in real setups
SLACK_WEBHOOK = "{{ var.value.slack_webhook }}"  # or use Airflow Connection via hook


def notify_slack(context):
    dag_id = context.get('dag').dag_id if context.get('dag') else "unknown"
    run_id = context.get('run_id')
    exception = context.get('exception')
    exec_date = context.get('execution_date')
    msg = {
        "text": f":red_circle: DAG *{dag_id}* failed.\nRun: {run_id}\nExecution date:
{exec_date}\nError: {exception}"
    }
    # Use requests inside callback; in production use safer auth and error handling
    webhook = os.environ.get("SLACK_WEBHOOK")  # set via env or use Airflow
Variable/Connection
    if webhook:
        try:
            requests.post(webhook, json=msg, timeout=10)
        except Exception as e:
            print("Failed to send Slack alert:", e)
    else:
        print("Slack webhook not configured; alert payload:", msg)


def failing_task():
    raise Exception("intentional failure to test alerts")


with DAG(
    dag_id="dag_with_failure_alerts",
    start_date=datetime(2025,1,1),
    schedule_interval=None,
    catchup=False,
    default_args={"retries": 0},
    on_failure_callback=notify_slack
) as dag:
    t1 = PythonOperator(task_id="failing_task", python_callable=failing_task)
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

31. A PythonOperator must pass a table name to a downstream BigQueryOperator. How would you implement this?

Step 1. Decide the handoff mechanism — for a small string like a table name, use XCom (task return or xcom_push) because it's simple and built-in.

Step 2. In the upstream task return the table name (or call ti.xcom_push("table", value)) so Airflow stores it in XCom automatically.

Step 3. In the downstream BigQuery operator reference the XCom via a Jinja template (e.g. {{ ti.xcom_pull(task_ids='prepare_table') }}) or pull it in a Python callable and pass to the operator.

Step 4. Prefer returning a canonical full table id (project.dataset.table) to avoid downstream ambiguity about default project/dataset.

Step 5. If using TaskFlow API, return the table name from the decorated function and accept the returned value as an argument in the downstream task (Airflow handles the XCom wiring).

Step 6. Add validation in the upstream task (exists/allowed pattern) and in the downstream task check that pulled table name is not None to fail fast.

Step 7. Keep the naming stable across retries by using run-specific identifiers (e.g., append {{ ds_nodash }}) if you generate temp tables per-run.

Step 8. Document the XCom key or return contract (which task pushes it and the expected format) so future maintainers know the contract.

Step 9. If multiple table names may be produced, push a dict or manifest and have downstream read the exact key it needs.

Step 10. For production robustness, add a small safety fallback: if XCom is empty, raise a clear error rather than silently continue.

Code (example using **PythonOperator** to push and **BigQueryInsertJobOperator** to consume):

```
from airflow import DAG

from airflow.operators.python import PythonOperator

from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator

from datetime import datetime


def prepare_table(**kwargs):

    # compute or choose table name

    table = "myproject.mydataset.temp_table_{{ ds_nodash }}"

    # push via return (Airflow stores return in XCom)

    return table


with DAG("pass_table_name_example", start_date=datetime(2025,1,1),
schedule_interval=None, catchup=False) as dag:
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
push_table = PythonOperator(
    task_id="prepare_table",
    python_callable=prepare_table,
    provide_context=True
)

# downstream reads XCom using Jinja templating
run_bq = BigQueryInsertJobOperator(
    task_id="run_bq",
    configuration={
        "query": {
            "query": "SELECT COUNT(1) as cnt FROM `{{ ti.xcom_pull(task_ids='prepare_table') }}` ",
            "useLegacySql": False
        }
    }
)

push_table >> run_bq
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

32. You must share a large dataset between tasks. Since XComs are not suitable, what alternative would you design?

Step 1. Persist the large dataset to a durable external store that both tasks can access: GCS (Parquet/CSV), BigQuery staging table, Cloud Storage + manifest, or a shared DB (Cloud SQL / Cloud Storage).

Step 2. Choose the best format: Parquet for columnar and compact storage, newline-delimited JSON/CSV for compatibility, or BigQuery table if you need SQL access and metadata.

Step 3. Upstream task writes the dataset to the chosen location and emits a small pointer (URI or table name) via XCom so downstream tasks know where to read.

Step 4. Make writes atomic from the pipeline's perspective: write to a temp path (e.g., .../tmp/<run_id>/) then rename/move to final path or write a "done" marker file to signal completeness.

Step 5. Downstream task reads from the pointer (GCS URI or BigQuery table) and validates the presence and schema before processing.

Step 6. For very large objects consider chunking / sharding and publish a manifest file listing shards; downstream reads manifest and processes shards in parallel.

Step 7. Add lifecycle/retention rules and a cleanup task to delete intermediate storage after successful downstream processing to control cost.

Step 8. Ensure IAM: both tasks' service account(s) must have read/write access to the chosen store (GCS buckets, BigQuery dataset, etc).

Step 9. Log metadata (rows, bytes, checksums) into an audit table so you can trace which run produced which files and re-run safely.

Step 10. If low-latency access is required, use a fast store (Cloud Storage + Parquet for batch; regional Cloud SQL or Redis for small-medium payloads).

Code example: upstream writes a DataFrame to GCS parquet and pushes the GCS URI; downstream reads it. Bolded operator/hook names: **GCSHook** (or use python libraries):

```
from airflow import DAG

from airflow.operators.python import PythonOperator

from airflow.providers.google.cloud.hooks.gcs import GCSHook

import pandas as pd

import pyarrow as pa

import pyarrow.parquet as pq

import tempfile

from datetime import datetime


def write_parquet_to_gcs(**kwargs):

    df = pd.DataFrame({"a":[1,2,3]})

    uri = f"gs://my-bucket/staging/data_{{{{ ds_nodash }}}}.parquet"

    # write locally then upload (simplest)

    with tempfile.NamedTemporaryFile(suffix=".parquet") as tmp:
```

© Shubham Wadekar

```
        table = pa.Table.from_pandas(df)
        pq.write_table(table, tmp.name)
        hook = GCSHook()
        hook.upload(bucket_name="my-bucket", object_name=f"staging/data_{{{{ ds_nodash }}}}.parquet", filename=tmp.name)
    return uri  # returned value stored in XCom


def read_parquet_from_gcs(**kwargs):
    ti = kwargs['ti']
    uri = ti.xcom_pull(task_ids='write_parquet_to_gcs')
    # use GCSHook to download then read with pyarrow/pandas (omitted for brevity)
    print("Read from", uri)


with DAG("large_dataset_sharing", start_date=datetime(2025,1,1), schedule_interval=None, catchup=False) as dag:
    write = PythonOperator(task_id="write_parquet_to_gcs", python_callable=write_parquet_to_gcs)
    read = PythonOperator(task_id="read_parquet_from_gcs", python_callable=read_parquet_from_gcs)
    write >> read
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

33. You want to track record counts across tasks. How would you store and share this information?

Step 1. For small numeric metrics (counts), XCom is fine — upstream pushes the integer (return or xcom_push) and downstream reads it.

Step 2. For durability, auditing and historical queries, write counts to an external metadata/audit table (BigQuery, Cloud SQL) with columns: run_id, task_id, record_count, timestamp, and any partitions.

Step 3. Upstream tasks should push both the count to XCom for immediate DAG logic and persist the same count to the audit store for long-term tracking.

Step 4. Use a small helper function or operator that standardizes how counts are recorded (single SQL insert into the audit table) so all tasks report consistently.

Step 5. If multiple tasks produce counts in parallel, have each insert a row keyed by (run_id, task_id) so an aggregator can later read and compute totals or diffs.

Step 6. For near-real-time dashboards, stream counts into a metrics system (Cloud Monitoring / Prometheus) in addition to the audit store.

Step 7. Use BigQuery or a time-series store if you want to run historical analysis, SLA checks, or alerting on anomalous counts.

Step 8. Add schema and retention policies for the audit table and periodically aggregate older rows into rollup tables if needed.

Step 9. Expose a small utility operator or Python callable to read counts (from XCom or audit table) and make branching decisions (e.g., abort if count < threshold).

Step 10. Secure the audit store with least-privilege IAM and document the metric names and meanings for downstream consumers.

Code example: push count via **PythonOperator** and record to BigQuery using **BigQueryInsertJobOperator**:

```
from airflow import DAG

from airflow.operators.python import PythonOperator

from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator

from datetime import datetime


def produce_count(**kwargs):

    # compute count

    count = 123

    # push to XCom by returning

    return count
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
def build_insert_query(run_id, task_id, count):
    return f"""
    INSERT INTO `myproj.metadata.task_counts`  (run_id, task_id, record_count, ts)
    VALUES ('{run_id}', '{task_id}', {count}, CURRENT_TIMESTAMP())
    """


with DAG("record_counts_example", start_date=datetime(2025,1,1), schedule_interval=None,
catchup=False) as dag:


    produce = PythonOperator(task_id="produce_count", python_callable=produce_count)


    # record to BigQuery using value pulled from XCom in the query template
    record = BigQueryInsertJobOperator(
        task_id="record_count_bq",
        configuration={
            "query": {
                "query": "{{ task_instance.xcom_pull(task_ids='produce_count') | string }}",  # simplified;
prefer building full INSERT via Python if complex
                "useLegacySql": False
            }
        }
    )


    produce >> record
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

34. A pipeline must run with different input paths for dev, test, and prod. How would you handle this?

Step 1. Decide the source of truth for environment: use either separate Composer environments (one per env) or a single Composer with a clear env identifier (ENV variable, Airflow Variable, or DAG param).

Step 2. Keep environment-specific values (GCS prefixes, buckets, table names) out of code — store them in Airflow Variables, Connections, or a secrets backend (Secret Manager) with a consistent naming convention like mypipeline.path.dev, mypipeline.path.test, mypipeline.path.prod.

Step 3. Make the DAG accept an env parameter (dag default or dag_run.conf) so you can override at runtime for manual runs; fall back to the composer environment name if not provided.

Step 4. Build a small config loader inside the DAG that reads the appropriate Variable/Secret based on env and returns a dictionary of paths; keep this loader reusable across DAGs.

Step 5. Use Jinja templating or pass resolved paths into operators as normal Python variables (avoid constructing paths inside operator templates where possible for clarity).

Step 6. Secure per-environment credentials via Connections/Secret Backends and ensure IAM roles differ appropriately between dev/test/prod to avoid accidental cross-env access.

Step 7. Document the mapping and provide a simple CLI/airflow command or UI guide for operators to trigger DAGs in each environment (or use CI to deploy DAGs with the correct env config).

Step 8. For CI/CD, keep environment config in the deployment pipeline (e.g., Terraform or GitHub Actions) and deploy the same DAG code to each Composer environment with environment-specific Variables already set.

Step 9. Add a runtime guard that validates the resolved path exists (or the bucket is reachable) before proceeding, and fail fast with a clear error if the env config is missing.

Step 10. If you need to test production logic against smaller sample data, allow a sample flag that points prod-configured DAG to a test prefix without changing production configs.

Example: resolve environment-specific input path using Airflow Variables and use it in an operator

Resolve env config and use in an operator:

```
from airflow import DAG

from airflow.operators.python import PythonOperator

from airflow.models import Variable

from datetime import datetime
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
def get_env_config(env):
    # Variables named like: "mypipeline.input.dev", "mypipeline.input.test",
"mypipeline.input.prod"
    key = f"mypipeline.input.{env}"
    path = Variable.get(key, default_var=None)
    if not path:
        raise ValueError(f"Missing Variable for env={env}: {key}")
    return {"input_path": path}


def process(**kwargs):
    env = kwargs.get("params", {}).get("env") or Variable.get("composer_env", "dev")
    cfg = get_env_config(env)
    input_path = cfg["input_path"]
    print("Using input path:", input_path)
    # pass input_path to real processing logic


with DAG("env_specific_paths", start_date=datetime(2025,1,1), schedule_interval=None,
catchup=False) as dag:
    run = PythonOperator(
        task_id="process",
        python_callable=process,
        provide_context=True,
        params={"env": "{{ dag_run.conf.get('env') if dag_run else '' }}"}  # allows runtime override
    )
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

35. A SQL query in BigQuery must run dynamically based on yesterday's date. How would you implement it in Composer?

Step 1. Use Airflow templating and execution context — ds is the DAG run logical date (YYYY-MM-DD); to get yesterday relative to ds, use macros.ds_add(ds, -1).

Step 2. Put the date expression inside the SQL or operator configuration using Jinja (e.g., {{ macros.ds_add(ds, -1) }}) so the query is rendered per-run.

Step 3. Prefer ds/execution_date semantics over datetime.now() so backfills and catchups behave predictably relative to scheduled logical dates.

Step 4. If you need a different date format (no dashes), use {{ macros.ds_add(ds, -1).replace('-', '') }} or {{ macros.ds_format(macros.ds_add(ds, -1), '%Y-%m-%d', '%Y%m%d') }}.

Step 5. Keep queries idempotent for retries: target a specific partition (e.g., mytable$YYYYMMDD) or use WHERE DATE(col) = '{{ macros.ds_add(ds, -1) }}'.

Step 6. Use **BigQueryInsertJobOperator** (or provider-specific operator) and put the templated SQL under configuration.query.query so it's rendered by Airflow before submission.

Step 7. Add pre-checks if necessary (e.g., verify source partition exists) and short-circuit the DAG if no data for that date.

Step 8. Parametrize the query further using DAG params or Variables if the date logic might change (e.g., run for business date vs UTC date).

Step 9. Log the resolved date in the DAG run metadata for easy debugging and auditing.

Step 10. Test the templated query rendering in a staging DAG run with different execution_date values (or using airflow dags trigger --conf) to confirm the date substitution.

Example: BigQuery query using yesterday's date via macros.ds_add

BigQueryInsertJobOperator usage:

```
from airflow import DAG

from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator

from datetime import datetime


with DAG("bq_yesterday_query", start_date=datetime(2025,1,1), schedule_interval="@daily",
catchup=False) as dag:


  run_query = BigQueryInsertJobOperator(

    task_id="run_yesterday_query",

    configuration={

      "query": {

        "query": """

        SELECT user_id, COUNT(*) as events

        FROM `myproj.streaming.raw_events`

        WHERE DATE(event_timestamp) = '{{ macros.ds_add(ds, -1) }}'
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
        GROUP BY user_id
        """,
        "useLegacySql": False
      }
    }
  )
```

36. Your workflow requires environment-specific parameters. How would you configure them in Composer?

Step 1. Store environment-specific parameters in a secure, centralized place: Airflow Variables for non-sensitive values and Secret Manager (via Airflow secrets backend) or Connections for credentials.

Step 2. Use a naming convention for variables and secrets (e.g., mypipeline.param.prod, mypipeline.param.test) so code can resolve them by env at runtime.

Step 3. Wire the Composer environment to a secret backend (Secret Manager, Hashicorp Vault) so secrets are not stored in the Airflow DB and access is audited.

Step 4. In the DAG, load parameters at runtime using Variable.get() or via the secrets backend — encapsulate this in a small config loader utility to keep DAGs clean.

Step 5. Allow runtime overrides via dag_run.conf for ad-hoc runs (useful for reprocessing or testing), and validate inputs before proceeding.

Step 6. Use Connections for service endpoints (e.g., http_conn_id, google_cloud_default) rather than baking credentials into DAG code; reference them in operators/hooks.

Step 7. For deployment, keep non-sensitive environment configs managed by your infra code (Terraform) so environments are reproducible and drift is minimized.

Step 8. Provide default fallbacks and fail-fast behavior: if a required parameter is missing for the env, raise a clear error rather than proceeding with unsafe defaults.

Step 9. Document the parameter schema (keys, types, required/optional) and include a small startup validation DAG that checks all required env parameters are present.

Step 10. For complex apps, consider a single config file per env stored in GCS or Secrets Manager (JSON/YAML) that the DAG downloads and validates at runtime; this makes adding new parameters simpler.

Example: config loader using Airflow Variables and Secret Manager fallback

Config loader + usage (Variable.get and Secret backend):

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.models import Variable
import os
from datetime import datetime
```

© Shubham Wadekar

```
def load_config(env):
    # prefer Airflow Variable
    key = f"mypipeline.config.{env}"
    cfg = Variable.get(key, default_var=None)
    if cfg:
        return cfg  # assume JSON string or similar parsed by caller

    # fallback to environment variable (or secret backend)
    fallback = os.environ.get(f"MYPipeline_CONFIG_{env.upper()}")
    if fallback:
        return fallback

    raise ValueError(f"No config found for env={env}")

def task_using_config(**kwargs):
    env = Variable.get("composer_env", "dev")
    cfg = load_config(env)
    print("Loaded config for", env, cfg)

with DAG("env_config_example", start_date=datetime(2025,1,1), schedule_interval=None,
catchup=False) as dag:
    t = PythonOperator(task_id="use_config", python_callable=task_using_config)
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

37. A task runs successfully, but you want to capture its execution time for reporting. How would you set this up?

Step 1. Decide where to store the metric — short-term for DAG logic (XCom) and long-term for reporting (BigQuery/Cloud Monitoring / a metrics DB).

Step 2. Capture timing in an on_success_callback (or inside the task code) using the task instance context — compute duration from ti.start_date and ti.end_date or use ti.duration if available.

Step 3. In the callback convert the duration to a standard unit (seconds or milliseconds) and enrich with metadata (dag_id, task_id, run_id, execution_date).

Step 4. Push the runtime to XCom for immediate downstream use if another task needs it within the same DAG run.

Step 5. Persist the metric to a durable store for historical reporting — e.g., insert a row into a BigQuery audit table or push a custom metric to Cloud Monitoring.

Step 6. If you push to Cloud Monitoring, use the Monitoring API / client library to emit a time series so dashboards/alerts can be defined easily.

Step 7. Add a small utility function or operator to standardize the capture+persist logic so every DAG can reuse it (avoid duplicated alerting code).

Step 8. Ensure the capture is robust: guard against missing start_date/end_date and tolerate small clock skew; record None or -1 if timing cannot be computed.

Step 9. Add a lightweight dashboard or SQL view (BigQuery) that aggregates durations by dag/task/run and exposes p50/p95 and trends for SRE/Business reporting.

Step 10. Test end-to-end: run a task in staging, verify XCom value, verify entry in BigQuery/Cloud Monitoring, and confirm dashboards pick it up.

on_success_callback capture duration (example):

```
# Example callback that captures a task's runtime and writes to BigQuery.
# Use this as dag-level or task-level on_success_callback.

from datetime import timezone
from google.cloud import bigquery
import json

def capture_duration_and_record(context):
    ti = context['ti']
    dag_id = context['dag'].dag_id if context.get('dag') else None
    task_id = ti.task_id
    run_id = context.get('run_id')
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
    # compute duration robustly
    start = ti.start_date
    end = ti.end_date
    if start and end:
        duration_secs = (end - start).total_seconds()
    else:
        # fallback to ti.duration if set, else None
        duration_secs = getattr(ti, 'duration', None)

    record = {
        "dag_id": dag_id,
        "task_id": task_id,
        "run_id": run_id,
        "execution_date": context.get('execution_date').isoformat() if context.get('execution_date')
else None,
        "duration_secs": duration_secs
    }

    # Example: write to BigQuery audit table
    try:
        client = bigquery.Client()
        table_id = "myproj.monitoring.task_durations"
        errors = client.insert_rows_json(table_id, [record])
        if errors:
            # optionally log the error somewhere durable
            print("BQ insert errors:", errors)
    except Exception as e:
        # fallback: print/log to Airflow logs
        print("Failed to record duration:", e)
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

38. A DAG fails in production, but the logs are too large to view in the UI. How would you troubleshoot it?

Step 1. Don't rely solely on the UI — fetch logs from the centralized logging backend (Cloud Logging/Stackdriver) or the worker's log storage (GCS, if configured) where full logs are retained.

Step 2. Use filtering to narrow the log volume: filter by dag_id, task_id, execution_date, severity, or specific error text to find the relevant slices quickly.

Step 3. Export or stream logs to BigQuery or GCS if not already done — that makes searching, grep/SQL, and sharing easier than the UI.

Step 4. Use the CLI/SDK to pull logs directly into your terminal where tools like grep, sed, less, and awk make triage faster. For GCP use gcloud logging read with filters.

Step 5. Pull compressed logs for the time window and inspect locally (download full worker log file from GCS), or use gcloud logging read with --limit and --format and then pipe into analysis tools.

Step 6. If logs are too verbose, reproduce the error in staging with increased log level only for the failing module or add temporary targeted debug statements to capture the failing context, then revert.

Step 7. Capture and persist additional diagnostics (stack traces, exception payloads, relevant task inputs like file paths or job_ids) in a small artifact (GCS file or BigQuery row) from on_failure_callback so you have one canonical failure payload to analyze.

Step 8. For multi-worker issues, correlate logs across components (scheduler, worker, Kubernetes pod, Cloud Logging for Dataflow/Dataproc) using job_ids/run_ids and timestamps — centralized logging with those fields makes correlation simple.

Step 9. If the UI truncates logs because of size, set log export in Airflow to write logs to GCS (or adjust remote_logging settings) so entire logs are preserved and conveniently downloadable.

Step 10. Automate a "fetch and summarize" helper (script or small CLI wrapper) that given dag_id/task_id/execution_date downloads the full logs, extracts relevant stack trace and prints the last N KB — this saves repeated manual steps.

Troubleshooting commands (GCP example):

```
# Read Cloud Logging entries for a specific DAG run (filter by labels or text)

gcloud logging read \

  'resource.type="cloud_composer_environment" AND jsonPayload.task_id="my_task" AND jsonPayload.dag_id="my_dag" AND timestamp >= "2025-09-10T00:00:00Z"' \

 --project=myproj --limit=1000 --format="json" > logs.json


# If Airflow is configured to put task logs in GCS, download the worker log file:

gsutil cp gs://my-composer-logs/bucket/<path-to-task-log>.log ./task.log

less ./task.log
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

Example: fetch last 10000 lines of a GCS-hosted log (streaming)

gsutil cat gs://my-composer-logs/... | tail -n 10000

39. A business wants Slack notifications for all failed jobs. How would you configure this?

Step 1. Centralize the Slack notification logic into a reusable callback function (notify_slack) so it can be applied DAG-level or registered globally.

Step 2. Store the Slack webhook (or use Slack app with OAuth token) as an Airflow Connection or Airflow Variable / Secret Manager — never hardcode secrets in DAGs.

Step 3. Use a DAG-level on_failure_callback (or set a default on_failure_callback in default_args used by all DAGs) so every failure triggers the same handler centrally.

Step 4. For global coverage, set default_args in your DAG factory or create a small wrapper that injects the on_failure_callback into every DAG you create; alternatively configure a global failure handler via plugins/airflow_local_settings if you control the environment.

Step 5. In the callback include useful context in the message: dag_id, task_id, execution_date, run_id, a short error message and a link to the UI logs (Airflow URL) and runbook link.

Step 6. To avoid alert storms, group multiple task failures in the same DAG run into a single Slack message — dedupe by run_id or aggregate failed tasks in the callback before sending.

Step 7. Use Slack rate-limits best practices (batch messages if many failures) and include quick-action buttons or links if you use a Slack app with interactivity.

Step 8. Test in staging by intentionally failing a DAG and verifying Slack receives the message with the expected content and links.

Step 9. Monitor the Slack notifications channel for volume and refine severity routing (critical DAGs -> PagerDuty, non-critical -> Slack channel) to avoid alert fatigue.

Step 10. Keep the notification code small and resilient — if Slack outage prevents sending, log the payload to a durable store (GCS/BigQuery) so alerts are not lost.

Slack failure notifier using SlackWebhookHook (example):

Example: reusable on_failure_callback that posts to Slack via SlackWebhookHook.

Put SLACK_WEBHOOK_URL in Airflow Variable or Connection and reference it here.

```
from airflow.providers.slack.hooks.slack_webhook import SlackWebhookHook

from airflow import DAG

from airflow.operators.python import PythonOperator

from datetime import datetime

import os
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
def notify_slack(context):
    dag_id = context.get('dag').dag_id if context.get('dag') else "unknown"
    task_id = context['task_instance'].task_id
    exec_date = context.get('execution_date')
    run_id = context.get('run_id')
    exception = context.get('exception')
    airflow_base = os.environ.get('AIRFLOW_BASE_URL', 'https://airflow.example.com')  # set env
var in env
    log_url = f"{airflow_base}/dags/{dag_id}/grid?execution_date={exec_date}"

    message = (
        f":red_circle: DAG *{dag_id}* failed\n"
        f">*Task*: {task_id}\n"
        f">*Run*: {run_id}\n"
        f">*Execution Date*: {exec_date}\n"
        f">*Error*: {exception}\n"
        f"<{log_url}|View logs>"
    )

    # read webhook from Airflow Variable or env (prefer Variable/Connection in production)
    webhook = os.environ.get("SLACK_WEBHOOK")  # or Variable.get("slack_webhook")
    if webhook:
        hook = SlackWebhookHook(http_conn_id=None, webhook_token=webhook,
message=message)
        try:
            hook.execute()
        except Exception as e:
            print("Failed to send Slack message:", e)
    else:
        print("Slack webhook not configured; message:", message)

# Example DAG using notify_slack as on_failure_callback
with DAG("example_with_slack_alerts", start_date=datetime(2025,1,1),
schedule_interval=None, catchup=False,
        default_args={"retries": 0}, on_failure_callback=notify_slack) as dag:
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
def fail_task():
    raise Exception("intentional failure")


t = PythonOperator(task_id="failing_task", python_callable=fail_task)
```

40. A DAG must version-control DAGs and promote them from dev → test → prod. How would you implement this?

Step 1. Choose a Git branching strategy (e.g., feature branches → develop (dev) → release (test) → main (prod)) and enforce it with branch protection rules so changes flow through environments via pull requests.

Step 2. Keep DAG code generic and environment-agnostic: read env-specific values from Variables/Connections/Secrets so the same DAG artifact can run in dev/test/prod without code changes.

Step 3. Store DAGs in a single repository with clear folder structure (e.g., dags/, tests/, infra/) and add CI checks that run static linting and unit tests on every push/PR.

Step 4. Use separate Composer environments for dev/test/prod (preferred) so deploying the same DAG to different GCS DAG-buckets isolates runtime behavior and permissions.

Step 5. Implement promotion as automated CI/CD: merge to develop → deploy to dev Composer; merge to release branch → deploy to test Composer; tag/release or merge to main → deploy to prod Composer (manual approval step before prod).

Step 6. Protect production with gated approvals: require code review, passing tests, and a manual approval step (or an automated policy check) before the pipeline copies DAGs into the production Composer DAGs bucket.

Step 7. Track versions: include Git commit hash or semantic version in deployed DAG metadata (e.g., an annotation or file) so runs can be traced to the exact code that produced them.

Step 8. Use infrastructure-as-code (Terraform) to provision Composer environments and set Variables/Connections consistently per environment; keep env-specific config out of the repo or in a separate secure store.

Step 9. Provide a rollback plan: keep previous artifact copies in a versioned storage area (or revert Git tag) and a documented rollback procedure to quickly restore the last known-good DAG set.

Step 10. Audit and monitor: log deployments, who promoted what and when, and surface this in an internal dashboard or BigQuery table so you can trace configuration drift or unexpected changes.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

41. A team wants automated DAG deployment using Cloud Build. How would you design the process?

Step 1. Trigger Cloud Build on Git events: create triggers for branches/tags (push to develop → deploy to dev; push to release → deploy to test; tag or merge to main → deploy to prod) so commits automatically start deployments.

Step 2. In Cloud Build, run pre-deploy steps: lint the Python (flake8/black), run unit tests (pytest for DAG logic), run a DAG import test (Airflow DAGBag import) to catch syntax/runtime import errors.

Step 3. Require a manual approval step for production deploys: use Cloud Build approvals or a separate manual promotion job to prevent accidental prod pushes.

Step 4. Deploy by syncing the built DAG artifacts to the Composer environment's DAGs GCS bucket (gsutil rsync or gcloud composer environments storage dags import) so Composer picks up new DAGs automatically.

Step 5. Make the Cloud Build config idempotent and safe: deploy to a staging prefix or bucket first, verify DAGs are healthy in test Composer (smoke-run), then copy to prod on approval.

Step 6. Secure credentials and secrets: store service account keys or use Cloud Build's service account with least privilege and grant it write access only to the target Composer DAG bucket(s).

Step 7. Emit deployment metadata: after deploy, write a small manifest (git SHA, build id, user, timestamp) to a deployment audit BigQuery table or GCS path for traceability.

Step 8. Add rollback and verification steps: post-deploy smoke tests (trigger a quick DAG run or import check) and, on failure, automatically revert the DAGs to the previous revision.

Step 9. Enforce policies via CI: ensure Cloud Build runs security linters, dependency checks, and license checks before allowing a deploy to proceed.

Step 10. Monitor and alert: notify the team on build success/failure via Slack or email, and surface long-running or failed deployments in dashboards.

Cloud Build example (cloudbuild.yaml):

```
# Cloud Build pipeline: lint -> unit tests -> smoke -> deploy to Composer DAG bucket

steps:

 # 1) Install dependencies

 - name: 'python:3.10'

   id: 'install-deps'

   entrypoint: 'bash'

   args:

     - -c

      - |

       pip install -r requirements.txt

       pip install apache-airflow  # for DAGBag import test (use pinned version matching
Composer)
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
# 2) Lint
- name: 'python:3.10'
  id: 'lint'
  entrypoint: 'bash'
  args:
    - -c
    - |
      flake8 dags || exit 1

# 3) Unit tests
- name: 'python:3.10'
  id: 'pytest'
  entrypoint: 'bash'
  args:
    - -c
    - |
      pytest -q

# 4) DAG import sanity check
- name: 'python:3.10'
  id: 'dag_import_test'
  entrypoint: 'bash'
  args:
    - -c
    - |
      python -c "from airflow.models import DagBag; bag=DagBag(dag_folder='dags',
include_examples=False);
            print('import_errors', bag.import_errors);
            assert not bag.import_errors"
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
 # 5) Deploy to Composer DAGs GCS bucket (dev/test/prod determined by trigger substitutions)
 - name: 'gcr.io/cloud-builders/gsutil'
   id: 'deploy'
   args:
     - '-m'
     - 'rsync'
     - '-r'
     - 'dags/'
     - 'gs://$COMPOSER_DAGS_BUCKET/'


substitutions:
  _COMPOSER_DAGS_BUCKET: ''  # set in trigger for dev/test/prod (or use
$COMPOSER_DAGS_BUCKET)
timeout: '1200s'
```

Notes about the example

- Use Cloud Build triggers with substitutions to set the correct COMPOSER_DAGS_BUCKET per environment.
- For production, add an approval step in Cloud Build UI or separate pipeline that runs only after manual approval.
- Use the Cloud Build service account with only the storage.objects.create and storage.objects.delete permissions on the target bucket.

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

42. How would you safely test DAGs in lower environments before promoting to production?

Step 1. Unit test DAG logic: write pytest tests to validate small pure-Python functions and operator parameter rendering; mock external calls where possible so tests are fast and deterministic.

Step 2. Run DAG import checks: ensure DagBag can import all DAG files without errors (catch syntax/import/runtime errors early).

Step 3. Smoke-run in dev Composer: deploy to a dev environment and run key DAGs with small synthetic data (use parameterized dag_run.conf to point to sample inputs) to verify end-to-end wiring.

Step 4. Use integration tests in test Composer: run DAGs against test datasets or test GCP projects that mimic production resources but with limited scale and cost controls.

Step 5. Use the TaskFlow API or unit-testable callables so heavy work can be exercised locally (pytest) without needing Composer — this speeds iteration.

Step 6. Apply canary deployments: deploy changes to a small subset of DAGs or run a single DAG with the new code in prod under a feature flag, verify behavior, then roll out.

Step 7. Automate post-deploy smoke tests: after each deploy to test, run a set of verification DAGs that assert expected artifacts (files, BigQuery rows, job statuses) exist and look correct.

Step 8. Use sandboxed credentials and least-privilege service accounts in non-prod so tests cannot accidentally touch production resources.

Step 9. Maintain test data/fixtures and replay tools: keep representative sample data and a replay DAG to reproduce tricky scenarios found in production.

Step 10. Capture and require sign-off: pipeline owners must review test run results (CI artifacts, logs, smoke test outputs) before manual approval gates allow promotion to production.

Quick commands / snippets for local & CI checks (examples):

DAG import check (local/unit):

```
python - <<'PY'
from airflow.models import DagBag
bag = DagBag(dag_folder='dags', include_examples=False)
if bag.import_errors:
  print('Import errors found:', bag.import_errors)
  raise SystemExit(1)
print('DAG import OK')
PY
```

Run a specific DAG task locally (for unit/integration during dev):

```
# Example: run a single Python task callable using pytest or python entrypoint
python -m pytest tests/test_my_dag.py -q
# or run callable directly in a small harness that sets up a fake context
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

43. You need to orchestrate a daily batch pipeline that loads GCS → BigQuery. How would you design the DAG?

Step 1. Define the contract: input GCS path pattern, expected schema, target BigQuery dataset/table and partitioning strategy (daily partition preferred).

Step 2. Trigger the DAG on a daily schedule or via GCS event + Pub/Sub trigger; use ds for run-date semantics so runs are reproducible.

Step 3. Add a lightweight pre-check that verifies files exist and are complete (GCS prefix existence or presence of a done/manifest file).

Step 4. Stage input into a stable staging location if needed (e.g., copy to gs://staging/<run>/), or load directly from the input path if atomic uploads are guaranteed.

Step 5. Load into a staging BigQuery table first (use a staging table per run) so schema issues and validation do not corrupt production data.

Step 6. Run data quality/validation checks on the staging table (row counts, nulls, schema conformance). Fail fast or branch based on results.

Step 7. Use an idempotent MERGE or partition-replace to move validated rows from staging into the production partitioned table (MERGE handles upserts).

Step 8. Archive or move processed files to an archive prefix (or delete) and emit a manifest/audit record (rows processed, run_id, runtimes) to a monitoring table.

Step 9. Configure retries, timeouts and use mode="reschedule" or deferrable sensors for any long waits to avoid occupying worker slots.

Step 10. Add observability: capture BigQuery job_ids, row counts, runtimes to BigQuery or Cloud Monitoring and alert on failures or SLA violations.

Code example (daily GCS → staging BQ → MERGE → archive). Operator names used in prose above are **GCSToBigQueryOperator**, **BigQueryInsertJobOperator**, **GCSToGCSOperator**.

```
from airflow import DAG

from airflow.operators.empty import EmptyOperator

from airflow.operators.python import PythonOperator

from airflow.providers.google.cloud.transfers.gcs_to_bigquery import GCSToBigQueryOperator

from airflow.providers.google.cloud.transfers.gcs_to_gcs import GCSToGCSOperator

from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator

from airflow.providers.google.cloud.sensors.gcs import
GCSObjectsWithPrefixExistenceSensor

from datetime import datetime, timedelta


PROJECT = "my-project"

STAGING_DATASET = "staging"

PROD_DATASET = "analytics"

INPUT_BUCKET = "ingest-bucket"

ARCHIVE_BUCKET = "archive-bucket"
```

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
default_args = {
    "owner": "data-eng",
    "depends_on_past": False,
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    "gcs_to_bq_daily",
    start_date=datetime(2025,1,1),
    schedule_interval="@daily",
    catchup=False,
    default_args=default_args,
) as dag:

    start = EmptyOperator(task_id="start")

    # Wait for at least one file for the run
    wait_for_files = GCSObjectsWithPrefixExistenceSensor(
        task_id="wait_for_files",
        bucket=INPUT_BUCKET,
        prefix="raw/{{ ds }}/",
        poke_interval=60,
        timeout=60 * 60 * 2,
        mode="reschedule",
    )

    # Load files into a run-specific staging table
    load_staging = GCSToBigQueryOperator(
        task_id="load_to_staging",
        bucket=INPUT_BUCKET,
        source_objects=["raw/{{ ds }}/**/*.csv"],
        destination_project_dataset_table=f"{PROJECT}.{STAGING_DATASET}.staged_{{{{
ds_nodash }}}}",
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
    source_format="CSV",
    write_disposition="WRITE_TRUNCATE",
    autodetect=True,
    skip_leading_rows=1,
)

# Validation: run simple row count check (could be extended)
validate = BigQueryInsertJobOperator(
    task_id="validate_staging",
    configuration={
        "query": {
            "query": """
            SELECT COUNT(1) AS cnt
            FROM `{project}.{stg}.staged_{ds_nodash}`
            """.format(project=PROJECT, stg=STAGING_DATASET, ds_nodash="{{ ds_nodash }}"),
            "useLegacySql": False,
        }
    },
)

# Merge validated staging into production partition (idempotent)
merge_to_prod = BigQueryInsertJobOperator(
    task_id="merge_to_prod",
    configuration={
        "query": {
            "query": """
            MERGE `{project}.{prod}.daily_table` T
            USING `{project}.{stg}.staged_{ds_nodash}` S
            ON T.id = S.id AND T._PARTITIONDATE = DATE('{{ ds }}')
            WHEN MATCHED THEN
              UPDATE SET T.col1 = S.col1, T.col2 = S.col2
            WHEN NOT MATCHED THEN
              INSERT (id, col1, col2, _PARTITIONDATE) VALUES (S.id, S.col1, S.col2, DATE('{{ ds }}'))
            """.format(project=PROJECT, prod=PROD_DATASET, stg=STAGING_DATASET),
            "useLegacySql": False,
```

© Shubham Wadekar

```
        }
    },
    retries=0,
)

# Archive processed files
archive = GCSToGCSOperator(
    task_id="archive_files",
    source_bucket=INPUT_BUCKET,
    source_object="raw/{{ ds }}/**",
    destination_bucket=ARCHIVE_BUCKET,
    destination_object="processed/{{ ds }}/",
    move_object=True,
)

end = EmptyOperator(task_id="end")

start >> wait_for_files >> load_staging >> validate >> merge_to_prod >> archive >> end
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

44. A hybrid pipeline requires Dataproc batch + Dataflow streaming + BigQuery. How would you orchestrate it?

Step 1. Separate concerns: let streaming (Dataflow) run continuously ingesting Pub/Sub into durable storage (BigQuery raw tables or partitioned GCS); treat Dataproc batch jobs as on-demand/periodic heavy transformations.

Step 2. Use Composer to manage lifecycle and coordination: start/stop Dataflow streaming jobs (if ephemeral), trigger Dataproc batch jobs, and kick off BigQuery transformations or merges.

Step 3. Define clear handoff artifacts: Dataflow writes to a partitioned raw BigQuery table or GCS; Dataproc reads those stable artifacts (or a subset) for heavy processing; output is written to staging BigQuery tables.

Step 4. Use XComs or persisted manifests to pass job metadata (Dataflow job_id, Dataproc job_id, output paths) between tasks for monitoring and troubleshooting.

Step 5. If Dataproc depends on up-to-date streaming data, include a readiness check (e.g., ensure streaming has emitted data up to a watermark) before starting batch work.

Step 6. Use **DataflowStartFlexTemplateOperator** or **DataflowStartTemplateOperator** to launch streaming jobs (or ensure they are running), and **DataprocCreateClusterOperator** + **DataprocSubmitPySparkJobOperator** (or Workflow Templates) to run batch jobs.

Step 7. Monitor jobs with sensors: **DataflowJobStatusSensor** for streaming job health and Dataproc job sensors (or use operator wait flags) for batch completion; capture logs/job messages on failures.

Step 8. After Dataproc job finishes, run BigQuery jobs (SQL MERGE/aggregation) using **BigQueryInsertJobOperator** to load or merge results into analytic tables.

Step 9. Implement retry policies and idempotent patterns for each component; if Dataproc job fails, optionally trigger a remediation DAG that reprocesses only affected partitions.

Step 10. Centralize observability: correlate Dataflow job_id, Dataproc job_id, and BigQuery load job_id in a monitoring/audit table and emit alerts on cross-system failures.

High-level DAG sketch (example orchestration). Operator names mentioned in prose above are **DataflowStartFlexTemplateOperator**, **DataprocCreateClusterOperator**, **DataprocSubmitPySparkJobOperator**, **BigQueryInsertJobOperator**.

```
from airflow import DAG

from airflow.operators.empty import EmptyOperator

from airflow.providers.google.cloud.operators.dataflow import
DataflowStartFlexTemplateOperator

from airflow.providers.google.cloud.sensors.dataflow import DataflowJobStatusSensor

from airflow.providers.google.cloud.operators.dataproc import (

    DataprocCreateClusterOperator,

    DataprocSubmitPySparkJobOperator,

    DataprocDeleteClusterOperator,

)
```

© Shubham Wadekar

```
from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator
from datetime import datetime, timedelta

PROJECT = "my-project"
REGION = "us-central1"
CLUSTER_NAME = "hybrid-cluster-{{ ds_nodash }}"

with DAG("hybrid_dataproc_dataflow_bq", start_date=datetime(2025,1,1),
schedule_interval="@daily", catchup=False) as dag:

  start = EmptyOperator(task_id="start")

  # Ensure streaming Dataflow job is running (idempotent start)
  start_stream = DataflowStartFlexTemplateOperator(
    task_id="start_streaming",
    project_id=PROJECT,
    region=REGION,
    body={
      "launchParameter": {
        "jobName": "streaming-{{ ds_nodash }}",
        "containerSpecGcsPath": "gs://my-templates/streaming-flex.json",
        "parameters": {"inputSubscription": "projects/myproj/subscriptions/raw-sub"}
      }
    },
  )

  # Optionally monitor streaming job health
  monitor_stream = DataflowJobStatusSensor(
    task_id="monitor_stream",
    project_id=PROJECT,
    region=REGION,
    job_id="{{ task_instance.xcom_pull(task_ids='start_streaming', key='job_id') }}",
    poke_interval=300,
    timeout=60 * 60 * 1,
  )
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
# Create ephemeral Dataproc cluster for batch transformations
create_cluster = DataprocCreateClusterOperator(
    task_id="create_cluster",
    project_id=PROJECT,
    cluster_name=CLUSTER_NAME,
    region=REGION,
    cluster_config={
        "master_config": {"num_instances": 1, "machine_type_uri": "n1-standard-4"},
        "worker_config": {"num_instances": 2, "machine_type_uri": "n1-standard-4"},
    },
)

# Submit PySpark job that reads streaming raw (BigQuery or GCS) and writes staging results
submit_spark = DataprocSubmitPySparkJobOperator(
    task_id="submit_spark",
    project_id=PROJECT,
    region=REGION,
    main="gs://my-bucket/jobs/process_streaming_output.py",
    cluster_name=CLUSTER_NAME,
    properties={"spark.submit.deployMode": "cluster"},
)

# Merge staging results into production BigQuery
merge_bq = BigQueryInsertJobOperator(
    task_id="merge_results",
    configuration={
        "query": {
            "query": """
            MERGE `myproj.analytics.final_table` T
            USING `myproj.staging.spark_output_{{ ds_nodash }}` S
            ON T.id = S.id
            WHEN MATCHED THEN UPDATE SET ...
            WHEN NOT MATCHED THEN INSERT ...
            """,
            "useLegacySql": False
```

© Shubham Wadekar

```
        }
    }
)

delete_cluster = DataprocDeleteClusterOperator(
    task_id="delete_cluster",
    project_id=PROJECT,
    cluster_name=CLUSTER_NAME,
    region=REGION,
    trigger_rule="all_done",
)

end = EmptyOperator(task_id="end")

start >> start_stream >> monitor_stream >> create_cluster >> submit_spark >> merge_bq
merge_bq >> delete_cluster >> end
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

45. You need to add data quality checks (nulls, duplicates) before loading data into BigQuery. How would you design this in Composer?

Step 1. Define DQ rules up front: which columns must be non-null, which columns must be unique (composite keys), acceptable value ranges, and minimum row counts.

Step 2. Centralize DQ checks as SQL templates or small Python callables so rules are versioned and reusable across DAGs.

Step 3. After loading data into a staging table, run a set of SQL checks using **BigQueryInsertJobOperator** or **BigQueryCheckOperator** that return pass/fail metrics (e.g., SELECT COUNT(1) FROM staging WHERE col IS NULL).

Step 4. Aggregate check results into a DQ results table (rows with rule_name, status, count, details) for auditing and trend analysis.

Step 5. Use a **BranchPythonOperator** (or decision task) to decide to proceed to production load only if critical checks pass; for non-critical checks allow proceed-with-warnings and surface alerts.

Step 6. For duplicates, use dedup SQL (ROW_NUMBER() OVER (PARTITION BY key ORDER BY updated_at) and keep first) or MERGE semantics to remove duplicates before merging into prod.

Step 7. Implement automatic remediation for simple fixes (e.g., coalesce nulls to defaults, cast types) and escalate complex failures for manual intervention.

Step 8. Add retries for transient failures, but avoid retrying checks indefinitely — treat DQ failures as data issues, not system issues.

Step 9. Emit DQ metrics to monitoring and send alerts for rule violations (Slack/email) and include links to sample failing rows (or to the failing file path).

Step 10. Test DQ rules with synthetic datasets in staging and maintain a runbook describing which checks are critical vs advisory.

Code example (run DQ checks via SQL and branch on results). Operator names referenced here are **BigQueryInsertJobOperator**, **BigQueryCheckOperator**, and **BranchPythonOperator**.

```
from airflow import DAG

from airflow.operators.python import BranchPythonOperator, PythonOperator

from airflow.providers.google.cloud.operators.bigquery import BigQueryInsertJobOperator

from airflow.operators.empty import EmptyOperator

from datetime import datetime


PROJECT = "my-project"

STAGING_TABLE = "my-project.staging.incoming_{{ ds_nodash }}"
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
def evaluate_checks(**kwargs):
    ti = kwargs['ti']
    # expects previous tasks to write JSON or XCom with check results
    results = ti.xcom_pull(task_ids='dq_checks_query')
    # Example expected results format: {"nulls": 0, "duplicates": 2, "row_count": 100}
    # Decide: if any critical check >0 then fail branch
    if not results:
        return 'dq_fail'
    if results.get('nulls', 0) > 0 or results.get('duplicates', 0) > 0:
        return 'dq_fail'
    return 'proceed_to_load'

with DAG("dq_before_load", start_date=datetime(2025,1,1), schedule_interval="@daily",
catchup=False) as dag:
    start = EmptyOperator(task_id="start")

    # run DQ check SQLs and write results to a small table or return via XCom.
    dq_checks_query = BigQueryInsertJobOperator(
        task_id="dq_checks_query",
        configuration={
            "query": {
                "query": f"""
                -- null check
                SELECT
                  SUM(CASE WHEN col1 IS NULL THEN 1 ELSE 0 END) AS nulls,
                  SUM(CASE WHEN dup_count > 1 THEN dup_count - 1 ELSE 0 END) AS duplicates,
                  COUNT(1) AS row_count
                FROM (
                  SELECT *, COUNT(1) OVER (PARTITION BY key_col) AS dup_count
                  FROM `{STAGING_TABLE}`
                )
                """,
                "useLegacySql": False,
            }
        },
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
    do_xcom_push=True,  # captures query result in XCom
)

decide = BranchPythonOperator(
    task_id="decide_on_dq",
    python_callable=evaluate_checks,
    provide_context=True,
)

dq_fail = PythonOperator(
    task_id="dq_fail",
    python_callable=lambda **kw: (_ for _ in ()).throw(Exception("DQ failed - aborting load"))
)

proceed = EmptyOperator(task_id="proceed_to_load")

# Example load step (MERGE into prod) - placeholder
load_prod = BigQueryInsertJobOperator(
    task_id="merge_into_prod",
    configuration={
      "query": {
        "query": """
        MERGE `my-project.analytics.final_table` T
        USING `{staging}` S
        ON T.id = S.id
        WHEN MATCHED THEN UPDATE SET ...
        WHEN NOT MATCHED THEN INSERT ...
        """.format(staging=STAGING_TABLE),
        "useLegacySql": False,
      }
    }
)

end = EmptyOperator(task_id="end", trigger_rule="none_failed_min_one_success")
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar

```
start >> dq_checks_query >> decide

decide >> [dq_fail, proceed]

proceed >> load_prod >> end

dq_fail >> end
```

© Shubham Wadekar

All rights reserved. Personal use only. Redistribution or resale is prohibited © Shubham Wadekar